Tangent-Space Methods for Nonlinear Control and Biomechanics

Volume 0: The Mathematical Primer

A Foundational Guide to Kinematics, Configuration, and State Space

Preface

Before we can begin to analyze the geometry of moving physical bodies or command them to navigate the world via control theory, we must establish a common language. The physics of movement happens in the real world, but our understanding of it happens entirely within mathematical spaces.

This primer, **Volume 0** of *Tangent-Space Methods for Nonlinear Control and Biomechanics*, is designed to bridge the gap between basic calculus/algebra and the rigorous differential geometry required for modern nonlinear control theory. By intuitively introducing State Space, Configuration Manifolds, SO(3) Rotations, and Screw Theory, this text ensures that the "lay user" possesses the foundational armor needed to attack the subsequent volumes on Tangent Spaces (Volume I) and Trajectory Control (Volume II).

The philosophy of this primer maintains our overarching theme: mathematics should be driven by the physical reality of motion, not abstract algebra for its own sake.

Contents

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{nx}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ **or** $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

$\mathbf{R}$ Control penalty matrix in LQR/DDP.

$\mathbf{K}$ Feedback gain matrix ($\delta\mathbf{u} = -\mathbf{K}\delta\mathbf{x}$).

$\boldsymbol{\gamma}$ A trajectory or flow, mapping time and initial conditions to states.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F_{muscle}}$ Force generated by muscles.

$\mathbf{W}$ Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

A Primer on Linear Algebra

> You cannot control a machine by staring at the machine. You control it by staring at the mathematical space that completely defines it.

In Plain Language 1.1. Context & Use Case: Why Linear Algebra is the Language of Motion

The Math You Will See: We will cover Vectors ($\mathbf{v}$), Matrices ($\mathbf{M}$), Vector Spaces and their axioms, Linear Independence and Basis, Eigenvalues/Eigenvectors ($\lambda, \mathbf{v}$), the Singular Value Decomposition (SVD), Quadratic Forms ($\mathbf{x}^T\mathbf{M}\mathbf{x}$), Positive Definiteness, and Dyads/Dyadics (the tensor products that underpin inertia, stress, and stiffness).
Why We Need It: A robot's joints or a car's velocity don't exist as single numbers; they exist as large arrays of interacting numbers. Linear algebra is the mathematics of these arrays. If you want to know if a drone will crash into the ground, you don't track every single rotor separately; you assemble the entire system into a single matrix and look at its eigenvalues.
All Variables Defined: Check the **Nomenclature** at the start of the book for standard vector and matrix conventions.

1.1 Historical Context

Linear algebra—the mathematics of vectors, matrices, and linear transformations—is arguably the single most consequential mathematical framework underpinning all of modern engineering. Its roots stretch back to ancient China (the *Nine Chapters on the Mathematical Art*, ca. 200 BCE, contained early row-reduction methods for solving systems of linear equations) and were formalized in the West through the work of Leibniz, Cramer, and Cayley in the 17th–19th centuries.

However, it was not until the mid-20th century that linear algebra assumed its central role in control theory. In 1960, Rudolf Kalman published his landmark paper introducing the *state space representation* of dynamical systems, which recast classical control problems—previously analyzed using frequency-domain transfer functions—into the language of matrices and vectors [**?**]. Kalman's formulation required engineers to reason about *controllability* and *observability* using rank conditions on matrices, placing linear algebra at the absolute foundation of modern control engineering.

Today, every simulation framework discussed in this text series—from MuJoCo's articulated-body integrator to Drake's trajectory optimization engine—performs thousands of matrix operations per millisecond. Understanding the structures, decompositions, and geometric interpretations of linear algebra is not optional; it is the prerequisite upon which every subsequent chapter of *The Geometry of Motion* is built.

1.2 Vectors, Matrices, and Vector Spaces

Before mapping physics, we must understand the fundamental data structures of control theory: vectors and matrices. More importantly, we must understand the abstract *space* in which these objects live, because the geometry of that space dictates everything about how physical systems behave. Readers seeking a comprehensive undergraduate treatment of these ideas will find Strang's textbook [**?**] an excellent companion to this chapter.

1.2.1 Vectors

A **vector** $\mathbf{v}$ is an ordered array of numbers. It can represent a position in space, a velocity profile, a list of sensor readings, or the activation levels of every muscle in a golfer's forearm. In physics, vectors implicitly assume a coordinate frame (e.g., X, Y, Z). A generic column vector in n-dimensional space is denoted:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n \tag{1.1}$$

It is crucial to distinguish two perspectives on what a vector "is." The *algebraic* perspective treats a vector as a column of numbers. The *geometric* perspective treats a vector as a directed arrow in space, possessing both magnitude and direction. Both perspectives are valid and useful; throughout this text, we will switch between them freely. When we write $\mathbf{v} \in \mathbb{R}^3$, we mean both the triple of numbers (v_1, v_2, v_3) and the arrow pointing from the origin to the point (v_1, v_2, v_3) in three-dimensional Euclidean space.

1.2.2 Matrices

A **matrix** $\mathbf{M}$ is a rectangular grid of numbers arranged in rows and columns. If a vector represents a point or a direction, a matrix acts as a set of instructions on how to *transform* that point—stretching it, rotating it, projecting it, or mapping it into an entirely different dimensional space. An $n \times m$ matrix maps vectors from $\mathbb{R}^m$ to $\mathbb{R}^n$:

$$\mathbf{M} = \begin{bmatrix} m_{11} & m_{12} & \cdots & m_{1m} \\ m_{21} & m_{22} & \cdots & m_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & \cdots & m_{nm} \end{bmatrix} \in \mathbb{R}^{n \times m} \tag{1.2}$$

When we multiply a matrix by a vector ($\mathbf{Mv}$), the result is a *linear combination of the columns of* $\mathbf{M}$, using the elements of $\mathbf{v}$ as weights. Specifically, if $\mathbf{m}_1, \mathbf{m}_2, \ldots, \mathbf{m}_m$ denote the

columns of $\mathbf{M}$, then:

$$\mathbf{M}\mathbf{v} = v_1\mathbf{m}_1 + v_2\mathbf{m}_2 + \cdots + v_m\mathbf{m}_m \tag{1.3}$$

This "column view" of matrix-vector multiplication is one of the most important conceptual tools in linear algebra. It means that the *range* (or image) of a matrix—the set of all possible outputs $\mathbf{M}\mathbf{v}$ as $\mathbf{v}$ varies over all of $\mathbb{R}^m$—is precisely the *span* of the columns of $\mathbf{M}$. Throughout this book, when we rotate a robot arm, we are physically applying a matrix to the vectors extending along its links.

1.2.3 The Axioms of a Vector Space

We have described vectors informally as "arrays of numbers." But the true power of linear algebra lies in the abstract concept of a **vector space**, which captures the essential algebraic properties that make vectors useful—regardless of whether the "vectors" are columns of numbers, functions, or even infinite-dimensional signals.

Definition 1.1. Vector Space

A *vector space* V over the real numbers $\mathbb{R}$ is a set of objects (called vectors) together with two operations—**addition** ($\mathbf{u}+\mathbf{v}$) and **scalar multiplication** ($\alpha\mathbf{v}$, where $\alpha \in \mathbb{R}$)—satisfying the following eight axioms for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and all $\alpha, \beta \in \mathbb{R}$:

(i) **Closure under addition:** $\mathbf{u}+\mathbf{v} \in V$.

(ii) **Commutativity:** $\mathbf{u}+\mathbf{v} = \mathbf{v}+\mathbf{u}$.

(iii) **Associativity of addition:** $(\mathbf{u}+\mathbf{v})+\mathbf{w} = \mathbf{u}+(\mathbf{v}+\mathbf{w})$.

(iv) **Existence of a zero vector:** There exists $\mathbf{0} \in V$ such that $\mathbf{v}+\mathbf{0} = \mathbf{v}$.

(v) **Existence of additive inverses:** For each $\mathbf{v} \in V$, there exists $-\mathbf{v} \in V$ such that $\mathbf{v}+(-\mathbf{v}) = \mathbf{0}$.

(vi) **Closure under scalar multiplication:** $\alpha\mathbf{v} \in V$.

(vii) **Distributivity:** $\alpha(\mathbf{u}+\mathbf{v}) = \alpha\mathbf{u}+\alpha\mathbf{v}$ and $(\alpha+\beta)\mathbf{v} = \alpha\mathbf{v}+\beta\mathbf{v}$.

(viii) **Compatibility:** $\alpha(\beta\mathbf{v}) = (\alpha\beta)\mathbf{v}$, and $1\cdot\mathbf{v} = \mathbf{v}$.

In Plain Language 1.2. Why Do We Care About Abstract Axioms?

You might wonder: why bother with these formal rules? Because the same mathematical structures appear over and over in radically different contexts. The set $\mathbb{R}^n$ of column vectors satisfies these axioms—but so does the set of all continuous functions $f : [0, T] \to \mathbb{R}$ (where "addition" means adding functions pointwise). In optimal control, the "vectors" we optimize over are often entire *trajectories*, which are functions of time. By proving a theorem about abstract vector spaces, we prove it simultaneously for column vectors, for functions, and for trajectories. The axioms are the price of generality.

Example 1.1. Verifying $\mathbb{R}^n$ is a Vector Space

Let us verify that $\mathbb{R}^n$ with standard component-wise addition and scalar multiplication satisfies the axioms.
Given $\mathbf{u} = (u_1, \ldots, u_n)$ and $\mathbf{v} = (v_1, \ldots, v_n)$ in $\mathbb{R}^n$, and $\alpha \in \mathbb{R}$:

- *Closure under addition:* $\mathbf{u} + \mathbf{v} = (u_1 + v_1, \ldots, u_n + v_n)$. Since each $u_i + v_i \in \mathbb{R}$, the sum is in $\mathbb{R}^n$. ✓
- *Zero vector:* $\mathbf{0} = (0, 0, \ldots, 0)$. Clearly $\mathbf{v} + \mathbf{0} = \mathbf{v}$. ✓
- *Additive inverse:* $-\mathbf{v} = (-v_1, \ldots, -v_n)$. Then $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$. ✓
- *Closure under scalar multiplication:* $\alpha\mathbf{v} = (\alpha v_1, \ldots, \alpha v_n) \in \mathbb{R}^n$. ✓

The remaining axioms (commutativity, associativity, distributivity, compatibility) follow immediately from the corresponding properties of real number arithmetic. Therefore $\mathbb{R}^n$ is a vector space. □

1.2.4 Linear Independence and Span

Definition 1.2. Linear Independence

A set of vectors $\{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k\}$ in a vector space V is called *linearly independent* if the only solution to the equation

$$\alpha_1\mathbf{v}_1 + \alpha_2\mathbf{v}_2 + \cdots + \alpha_k\mathbf{v}_k = \mathbf{0} \tag{1.4}$$

is the trivial solution $\alpha_1 = \alpha_2 = \cdots = \alpha_k = 0$. If a nontrivial solution exists (i.e., at least one $\alpha_i \neq 0$), the vectors are *linearly dependent*.

Intuitively, a set of vectors is linearly independent if none of them can be "built" from the others. In $\mathbb{R}^2$, two vectors are linearly independent if and only if they do not point in the same (or exactly opposite) direction. In $\mathbb{R}^3$, three vectors are linearly independent if and only if they do not all lie in the same plane.

Example 1.2. Linear Independence in $\mathbb{R}^3$

Consider the vectors:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{v}_3 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \tag{1.5}$$

Are these linearly independent? We solve $\alpha_1\mathbf{v}_1 + \alpha_2\mathbf{v}_2 + \alpha_3\mathbf{v}_3 = \mathbf{0}$:

$$\alpha_1 + \alpha_3 = 0 \tag{1.6}$$
$$\alpha_2 + \alpha_3 = 0 \tag{1.7}$$
$$0 = 0 \tag{1.8}$$

The third equation provides no constraint. Setting $\alpha_3 = 1$ yields $\alpha_1 = -1$, $\alpha_2 = -1$. A nontrivial solution exists: $-\mathbf{v}_1 - \mathbf{v}_2 + \mathbf{v}_3 = \mathbf{0}$, confirming that $\mathbf{v}_3 = \mathbf{v}_1 + \mathbf{v}_2$. The vectors are linearly *dependent*. Geometrically, all three vectors lie in the xy-plane, and the third is the diagonal of the rectangle formed by the first two.

1.2.5 Basis and Dimension

Definition 1.3. Basis

A *basis* for a vector space V is a set of vectors $\mathcal{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_n\}$ that is both linearly independent and spans V (i.e., every vector in V can be written as a linear combination of the basis vectors). The number n of vectors in any basis is called the *dimension* of V, written $\dim(V) = n$.

A fundamental theorem of linear algebra states that *every basis for a given vector space has exactly the same number of elements*. This is why "dimension" is well-defined—it is an intrinsic property of the space, not of any particular choice of basis.

The standard basis for $\mathbb{R}^n$ consists of the unit coordinate vectors $\mathbf{e}_1 = (1, 0, \ldots, 0)^T$, $\mathbf{e}_2 = (0, 1, 0, \ldots, 0)^T$, through $\mathbf{e}_n = (0, \ldots, 0, 1)^T$. Any vector $\mathbf{v} = (v_1, \ldots, v_n)^T$ can be written uniquely as $\mathbf{v} = v_1\mathbf{e}_1 + \cdots + v_n\mathbf{e}_n$. The numbers $v_1, \ldots, v_n$ are the *coordinates* of $\mathbf{v}$ with respect to this basis.

In Plain Language 1.3. Why Basis Matters for Robotics

When we model a robot arm, the "natural" basis might be the joint angles $(\theta_1, \theta_2, \ldots, \theta_n)$. But the task might require us to think in Cartesian coordinates (x, y, z). Changing basis—expressing the same physical motion in different coordinate systems—is one of the most common operations in robotics. The Jacobian matrix (Chapter 3) is precisely the mathematical tool that translates between these bases.

1.2.6 Linear Transformations

Definition 1.4. Linear Transformation

A function $T : V \to W$ between two vector spaces is called a *linear transformation* (or *linear map*) if it preserves vector addition and scalar multiplication:

(i) $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$ for all $\mathbf{u}, \mathbf{v} \in V$.

(ii) $T(\alpha\mathbf{v}) = \alpha\, T(\mathbf{v})$ for all $\alpha \in \mathbb{R}$ and $\mathbf{v} \in V$.

The fundamental connection between linear transformations and matrices is this: *every linear transformation between finite-dimensional vector spaces can be represented by a matrix, and every matrix represents a linear transformation.* Specifically, if $T : \mathbb{R}^m \to \mathbb{R}^n$ is linear,

then there exists a unique $n \times m$ matrix $\mathbf{M}$ such that $T(\mathbf{v}) = \mathbf{M}\mathbf{v}$ for all $\mathbf{v} \in \mathbb{R}^m$. The columns of $\mathbf{M}$ are simply $T(\mathbf{e}_1), T(\mathbf{e}_2), \ldots, T(\mathbf{e}_m)$—the images of the standard basis vectors.

Theorem 1.1. Matrix Multiplication is Linear

Let $\mathbf{M} \in \mathbb{R}^{n \times m}$. Then the function $T(\mathbf{v}) = \mathbf{M}\mathbf{v}$ is a linear transformation from $\mathbb{R}^m$ to $\mathbb{R}^n$.

Proof. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$ and $\alpha \in \mathbb{R}$. By the distributive property of matrix multiplication:

$$T(\mathbf{u} + \mathbf{v}) = \mathbf{M}(\mathbf{u} + \mathbf{v}) = \mathbf{M}\mathbf{u} + \mathbf{M}\mathbf{v} = T(\mathbf{u}) + T(\mathbf{v}) \tag{1.9}$$

and by the compatibility of matrix multiplication with scalar multiplication:

$$T(\alpha\mathbf{v}) = \mathbf{M}(\alpha\mathbf{v}) = \alpha(\mathbf{M}\mathbf{v}) = \alpha\, T(\mathbf{v}) \tag{1.10}$$

Both conditions of linearity are satisfied. □ □

1.2.7 Rank, Null Space, and the Rank-Nullity Theorem

Not all linear transformations preserve full information about the input. Some transformations "collapse" certain directions to zero.

Definition 1.5. Rank

The *rank* of a matrix $\mathbf{M}$, denoted $\text{rank}(\mathbf{M})$, is the number of linearly independent columns (equivalently, the number of linearly independent rows) of $\mathbf{M}$. It equals the dimension of the column space (the range of the corresponding linear transformation).

If a 3×3 matrix has rank 2, it collapses full 3D space into a 2D plane. In robotics, the rank of the Jacobian matrix $\mathbf{J}(\mathbf{q})$ tells you how many independent directions the end-effector can instantaneously move in. If the rank drops below its maximum value, the robot is at a *singularity*—it has lost the ability to move in at least one Cartesian direction, regardless of how its motors are commanded.

Definition 1.6. Null Space (Kernel)

The *null space* (or *kernel*) of a matrix $\mathbf{M}$, denoted $\mathcal{N}(\mathbf{M})$, is the set of all vectors that the matrix maps to zero:

$$\mathcal{N}(\mathbf{M}) = \{\mathbf{x} \in \mathbb{R}^m \mid \mathbf{M}\mathbf{x} = \mathbf{0}\} \tag{1.11}$$

For a humanoid robot, the null space of the arm's Jacobian describes all "internal motions" the arm can execute (like repositioning the elbow) while keeping the hand perfectly stationary holding a cup of coffee. These null-space motions are invisible in task space but consume real joint velocities. Exploiting them is a key strategy in redundant robot control.

Definition 1.7. The Determinant (Volumetric Interpretation)

For a square matrix $\mathbf{M} \in \mathbb{R}^{n\times n}$, the *determinant* $\det(\mathbf{M})$ is a scalar that measures the factor by which $\mathbf{M}$ scales n-dimensional volume. Imagine a unit hypercube $[0,1]^n$. Applying $\mathbf{M}$ to every point in that cube warps it into a parallelepiped whose signed volume equals $\det(\mathbf{M})$.

- If $\det(\mathbf{M}) = 1$, volume is perfectly preserved (like a pure rotation in space).
- If $\det(\mathbf{M}) = 0$, volume is crushed completely flat—the matrix is *singular* and loses rank. The transformation is not invertible.
- If $\det(\mathbf{M}) < 0$, volume is preserved in magnitude but orientation is flipped (a mirror reflection).

Now we arrive at one of the most important structural results in linear algebra:

Theorem 1.2. Rank-Nullity Theorem

For any $n \times m$ matrix $\mathbf{M}$:

$$\underbrace{\operatorname{rank}(\mathbf{M})}_{\text{dimension of range}} + \underbrace{\dim\big(\mathcal{N}(\mathbf{M})\big)}_{\text{dimension of null space}} = m \quad \text{(number of columns)} \tag{1.12}$$

Proof. Let $r = \operatorname{rank}(\mathbf{M})$ and let $\{\mathbf{n}_1, \ldots, \mathbf{n}_k\}$ be a basis for $\mathcal{N}(\mathbf{M})$, where $k = \dim(\mathcal{N}(\mathbf{M}))$. Extend this to a basis $\{\mathbf{n}_1, \ldots, \mathbf{n}_k, \mathbf{w}_1, \ldots, \mathbf{w}_{m-k}\}$ for all of $\mathbb{R}^m$ (this extension is always possible by a standard result in linear algebra).

We claim that $\{\mathbf{M}\mathbf{w}_1, \ldots, \mathbf{M}\mathbf{w}_{m-k}\}$ is a basis for the column space of $\mathbf{M}$, which would establish $r = m - k$, i.e., $r + k = m$.

Spanning: Any $\mathbf{y}$ in the range of $\mathbf{M}$ can be written as $\mathbf{y} = \mathbf{M}\mathbf{v}$ for some $\mathbf{v} \in \mathbb{R}^m$. Expanding $\mathbf{v}$ in our basis: $\mathbf{v} = \sum_{i=1}^{k} \alpha_i \mathbf{n}_i + \sum_{j=1}^{m-k} \beta_j \mathbf{w}_j$. Since $\mathbf{M}\mathbf{n}_i = \mathbf{0}$, we get $\mathbf{y} = \sum_{j=1}^{m-k} \beta_j \mathbf{M}\mathbf{w}_j$. So the $\mathbf{M}\mathbf{w}_j$ span the range.

Independence: Suppose $\sum_{j=1}^{m-k} \beta_j \mathbf{M}\mathbf{w}_j = \mathbf{0}$. Then $\mathbf{M}\left(\sum_j \beta_j \mathbf{w}_j\right) = \mathbf{0}$, meaning $\sum_j \beta_j \mathbf{w}_j \in \mathcal{N}(\mathbf{M})$. So $\sum_j \beta_j \mathbf{w}_j = \sum_{i=1}^{k} \gamma_i \mathbf{n}_i$ for some scalars γ_i. But $\{\mathbf{n}_1, \ldots, \mathbf{n}_k, \mathbf{w}_1, \ldots, \mathbf{w}_{m-k}\}$ is a basis for $\mathbb{R}^m$, so these vectors are linearly independent. Therefore all $\beta_j = 0$ and all $\gamma_i = 0$.

Thus $r = m - k$, establishing $\operatorname{rank}(\mathbf{M}) + \dim(\mathcal{N}(\mathbf{M})) = m$. □ □

In Plain Language 1.4. The Physical Meaning of Rank-Nullity

For a robot with m motors and an end-effector moving in n-dimensional task space:

- The **rank** of the Jacobian tells you how many independent task-space directions the robot can control.
- The **nullity** (dimension of the null space) tells you how many "extra" internal motions remain after the task is specified.

If a 7-DOF arm ($m = 7$) controls a hand in 3D position space ($n = 3$), and the Jacobian has rank 3, then the nullity is $7 - 3 = 4$. The robot has 4 degrees of internal freedom to reposition its elbow, shoulder, and wrist without moving the hand at all.

1.3 The Dot Product, Cross Product, and Inner Product Spaces

Two fundamental operations exist for combining vectors. Understanding them geometrically is essential for everything from computing kinetic energy to defining stability metrics.

1.3.1 The Dot Product (Inner Product)

The dot product (or inner product) takes two vectors and returns a scalar. It measures how much two vectors "align" with each other.

Definition 1.8. Dot Product

For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, the *dot product* is defined as:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \sum_{i=1}^{n} a_i b_i \tag{1.13}$$

In $\mathbb{R}^3$, this expands to $a_x b_x + a_y b_y + a_z b_z$.

The geometric interpretation connects the algebraic formula to angles:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos(\theta) \tag{1.14}$$

where θ is the angle between $\mathbf{a}$ and $\mathbf{b}$, and $\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}}$ is the Euclidean norm (length) of $\mathbf{a}$.

Derivation of Equation (1.14). Consider two vectors $\mathbf{a}$ and $\mathbf{b}$ with an angle θ between them. The vector $\mathbf{a} - \mathbf{b}$ forms the third side of a triangle. By the law of cosines:

$$\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2 \, \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta \tag{1.15}$$

Expanding the left side using the definition of the norm:

$$\begin{aligned} \|\mathbf{a} - \mathbf{b}\|^2 &= (\mathbf{a} - \mathbf{b})^T (\mathbf{a} - \mathbf{b}) && (1.16) \\ &= \mathbf{a}^T \mathbf{a} - 2\mathbf{a}^T \mathbf{b} + \mathbf{b}^T \mathbf{b} && (1.17) \\ &= \|\mathbf{a}\|^2 - 2\,\mathbf{a}^T \mathbf{b} + \|\mathbf{b}\|^2 && (1.18) \end{aligned}$$

Equating the two expressions:

$$\|\mathbf{a}\|^2 - 2\,\mathbf{a}^T \mathbf{b} + \|\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2 \, \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta \tag{1.19}$$

Canceling $\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2$ from both sides and dividing by -2:

$$\mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta \quad \square \tag{1.20}$$

$\square$

Key consequences of the dot product formula:

- If $\theta = 0$ (parallel vectors): $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\|$ (maximum positive value).
- If $\theta = 90°$ (perpendicular vectors): $\mathbf{a} \cdot \mathbf{b} = 0$. This is the fundamental test for **orthogonality**.
- If $\theta = 180°$ (antiparallel vectors): $\mathbf{a} \cdot \mathbf{b} = -\|\mathbf{a}\| \, \|\mathbf{b}\|$ (maximum negative value).

In control theory, the dot product appears constantly: kinetic energy is $E = \frac{1}{2}\dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q})\dot{\mathbf{q}}$, optimal control costs are $J = \mathbf{x}^T\mathbf{Q}\mathbf{x} + \mathbf{u}^T\mathbf{R}\mathbf{u}$, and projecting forces onto joint axes uses $\tau_i = \mathcal{F}^T \mathcal{S}_i$. In each case, a dot product computes a scalar "alignment" between a vector and a reference direction.

Orthogonal Projection

A common operation is projecting one vector onto another. The **scalar projection** of $\mathbf{b}$ onto $\mathbf{a}$ is:

$$\text{proj}_{\mathbf{a}}\mathbf{b} = \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a} \cdot \mathbf{a}} \, \mathbf{a} = \frac{\mathbf{a}^T\mathbf{b}}{\|\mathbf{a}\|^2} \, \mathbf{a} \tag{1.21}$$

The residual $\mathbf{b} - \text{proj}_{\mathbf{a}}\mathbf{b}$ is orthogonal to $\mathbf{a}$. This decomposition—splitting a vector into a component along a direction and a component perpendicular to it—is the geometric core of the Gram-Schmidt process and underlies the QR decomposition used in numerical linear algebra.

1.3.2 The Cross Product and the Skew-Symmetric Matrix

The cross product is a vector operation unique to three dimensions. It takes two vectors and generates a third vector that is perfectly perpendicular to both of them.

Definition 1.9. Cross Product

For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$, the *cross product* is:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{bmatrix} \tag{1.22}$$

The direction of $\mathbf{a} \times \mathbf{b}$ is determined by the *right-hand rule*: curl the fingers of your right hand from $\mathbf{a}$ toward $\mathbf{b}$; your thumb points in the direction of the cross product. The magnitude satisfies:

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \, \|\mathbf{b}\| \sin(\theta) \tag{1.23}$$

which equals the area of the parallelogram spanned by $\mathbf{a}$ and $\mathbf{b}$.

The cross product is *anti-commutative*: $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$. This has profound physical consequences. If you apply a force $\mathbf{F}$ at a displacement $\mathbf{r}$ from a pivot, the resulting torque is $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$. Reversing the order reverses the torque direction.

Crucially, the cross product can be rewritten as a matrix multiplication using the *skew-symmetric matrix*:

$$[\mathbf{a}_\times] = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} \quad \Longrightarrow \quad \mathbf{a} \times \mathbf{b} = [\mathbf{a}_\times]\mathbf{b} \tag{1.24}$$

Proposition 1.1. *The skew-symmetric matrix* $[\mathbf{a}_\times]$ *satisfies:*

(i) $[\mathbf{a}_\times]^T = -[\mathbf{a}_\times]$ *(skew-symmetry; equivalently, all diagonal entries are zero and the off-diagonal entries are negated across the diagonal).*

(ii) $\mathbf{a}^T[\mathbf{a}_\times]\mathbf{b} = 0$ *for all* $\mathbf{b}$ *(the cross product is always perpendicular to both input vectors).*

(iii) The eigenvalues of $[\mathbf{a}_\times]$ *are* $\{0, +i\|\mathbf{a}\|, -i\|\mathbf{a}\|\}$ *(purely imaginary, reflecting the rotational nature of the cross product).*

Proof. (i) is immediate from inspection of Equation (1.24): transposing negates every off-diagonal element while the diagonal remains zero.

(ii) We compute $\mathbf{a}^T([\mathbf{a}_\times]\mathbf{b}) = \mathbf{a}^T(\mathbf{a} \times \mathbf{b})$. Since $\mathbf{a} \times \mathbf{b}$ is perpendicular to $\mathbf{a}$ (by the geometric definition of the cross product), the dot product is zero.

(iii) The characteristic polynomial $\det([\mathbf{a}_\times] - \lambda\mathbf{I}) = -\lambda^3 - \|\mathbf{a}\|^2\lambda = -\lambda(\lambda^2 + \|\mathbf{a}\|^2)$. The roots are $\lambda = 0$ and $\lambda = \pm i\|\mathbf{a}\|$. □ □

This skew-symmetric matrix representation is absolutely pivotal for Chapters **??** and **??**, where we show that the Lie Algebra $\mathfrak{so}(3)$—the tangent space to the rotation group—consists entirely of 3×3 skew-symmetric matrices. Angular velocities live in this space.

The Scalar Triple Product

The scalar triple product $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$ computes the signed volume of the parallelepiped formed by three vectors $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$. It can be computed as a determinant:

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \det \begin{bmatrix} a_x & b_x & c_x \\ a_y & b_y & c_y \\ a_z & b_z & c_z \end{bmatrix} \tag{1.25}$$

If this determinant is zero, the three vectors are coplanar and linearly dependent—a direct connection between the cross product and the rank of the matrix formed by stacking the vectors as columns.

1.4 Eigenvalues, Eigenvectors, and Quadratic Forms

When you multiply a general vector by a matrix, the resulting vector typically changes both its magnitude and its direction. However, for almost every square matrix, there exist a few special directions that do *not* change their orientation when the matrix acts upon them. They are only stretched, shrunk, or flipped. These invariant directions reveal the deepest structural properties of the linear transformation.

1.4.1 The Eigenvalue Equation

Definition 1.10. Eigenvalues and Eigenvectors

Given a square matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$, a nonzero vector $\mathbf{v} \in \mathbb{R}^n$ is called an *eigenvector* of $\mathbf{M}$ if there exists a scalar λ (possibly complex) such that:

$$\mathbf{M}\mathbf{v} = \lambda \mathbf{v} \tag{1.26}$$

The scalar λ is the corresponding *eigenvalue*. The requirement that $\mathbf{v} \neq \mathbf{0}$ is essential; otherwise the equation would be trivially satisfied for any λ.

Deriving the Characteristic Polynomial

How do we actually find the eigenvalues? We rearrange Equation (1.26):

$$\mathbf{M}\mathbf{v} - \lambda \mathbf{v} = \mathbf{0} \quad \Longrightarrow \quad (\mathbf{M} - \lambda \mathbf{I})\mathbf{v} = \mathbf{0} \tag{1.27}$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. For a nonzero solution $\mathbf{v}$ to exist, the matrix $(\mathbf{M} - \lambda \mathbf{I})$ must be singular. By our earlier discussion, this means its determinant must vanish:

$$\det(\mathbf{M} - \lambda \mathbf{I}) = 0 \tag{1.28}$$

This is the **characteristic equation**. Expanding the determinant produces a polynomial of degree n in λ, called the **characteristic polynomial**. By the Fundamental Theorem of Algebra, this polynomial has exactly n roots (counted with multiplicity, possibly complex), giving us n eigenvalues.

Example 1.3. Computing Eigenvalues of a 2×2 Matrix

Consider the dynamics matrix of a simple spring system:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -4 & -5 \end{bmatrix} \tag{1.29}$$

Step 1: Form $\mathbf{A} - \lambda \mathbf{I}$:

$$\mathbf{A} - \lambda \mathbf{I} = \begin{bmatrix} -\lambda & 1 \\ -4 & -5 - \lambda \end{bmatrix} \tag{1.30}$$

Step 2: Compute the determinant and set it to zero:

$$\det(\mathbf{A} - \lambda \mathbf{I}) = (-\lambda)(-5 - \lambda) - (1)(-4) \tag{1.31}$$

$$= \lambda^2 + 5\lambda + 4 \tag{1.32}$$

$$= (\lambda + 1)(\lambda + 4) = 0 \tag{1.33}$$

Step 3: The eigenvalues are $\lambda_1 = -1$ and $\lambda_2 = -4$. Both are negative, meaning this system is **asymptotically stable**—any perturbation decays exponentially back to equilibrium. The mode associated with $\lambda_1 = -1$ decays slowly (time constant $\tau_1 = 1$ second), while the mode associated with $\lambda_2 = -4$ decays four times faster ($\tau_2 = 0.25$ seconds).

Step 4: Find the eigenvectors. For $\lambda_1 = -1$, solve $(\mathbf{A} + \mathbf{I})\mathbf{v}_1 = \mathbf{0}$:

$$\begin{bmatrix} 1 & 1 \\ -4 & -4 \end{bmatrix} \mathbf{v}_1 = \mathbf{0} \quad \Longrightarrow \quad \mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \tag{1.34}$$

For $\lambda_2 = -4$, solve $(\mathbf{A} + 4\mathbf{I})\mathbf{v}_2 = \mathbf{0}$:

$$\begin{bmatrix} 4 & 1 \\ -4 & -1 \end{bmatrix} \mathbf{v}_2 = \mathbf{0} \quad \Longrightarrow \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ -4 \end{bmatrix} \tag{1.35}$$

1.4.2 Physical Interpretation of Eigenvalues

1. **Inertia tensors and principal axes.** If $\mathbf{M}$ represents the mass-and-inertia tensor of a tumbling satellite, its eigenvectors identify the **Principal Axes**—the natural directions where the satellite spins stably. The eigenvalues give the moments of inertia about those axes. The famous "tennis racket theorem" (or Dzhanibekov effect) states that rotation about the axis with the *intermediate* eigenvalue is unstable, while rotation about the axes with the largest and smallest eigenvalues is stable.

2. **Stability of dynamical systems.** If $\mathbf{A}$ is the linearized dynamics matrix of a control system ($\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$), the eigenvalues dictate stability:

   - $\text{Re}(\lambda_i) < 0$ for all i: the system is **asymptotically stable** (all perturbations decay).
   - $\text{Re}(\lambda_i) > 0$ for any i: the system is **unstable** (perturbations grow exponentially).
   - $\text{Re}(\lambda_i) = 0$: the system is **marginally stable** (oscillations persist forever without growing or decaying).

   Eigenvalues are the fundamental heartbeat of linear dynamics. Every analysis in Chapter 2 builds on this fact.

Example 1.4. Visualizing Eigenvectors: The Stretching Rubber Sheet

Imagine printing a perfect circle onto a rubber square. You grab the edges of the rubber and stretch it wildly in various directions. The circle distorts into an ellipse.
Most straight lines drawn on the original circle have been both rotated and elongated. However, exactly two perpendicular lines—the major and minor axes of the resulting ellipse—did not rotate at all; they were only stretched. These non-rotating axes are your **eigenvectors**. The factors by which those specific lines were stretched (perhaps one stretched by $3\times$ and the other shrunk to $0.5\times$) are your **eigenvalues**. The matrix $\mathbf{M}$ is the algebraic instruction set for that stretching operation.
In n dimensions, a symmetric matrix transforms a hypersphere into a hyperellipsoid. The eigenvectors point along the principal axes of the ellipsoid, and the eigenvalues are the semi-axis lengths.

1.4.3 The Spectral Theorem for Symmetric Matrices

Among all matrices, *symmetric* matrices ($\mathbf{M} = \mathbf{M}^T$) occupy a privileged position. They arise naturally throughout mechanics (mass matrices, stiffness matrices, inertia tensors) and control theory (cost matrices, Lyapunov functions).

Theorem 1.3. The Spectral Theorem

If $\mathbf{M} \in \mathbb{R}^{n \times n}$ is symmetric ($\mathbf{M} = \mathbf{M}^T$), then:

(i) All eigenvalues of $\mathbf{M}$ are real.

(ii) Eigenvectors corresponding to distinct eigenvalues are orthogonal.

(iii) $\mathbf{M}$ can be factored as $\mathbf{M} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$, where $\mathbf{Q}$ is an orthogonal matrix ($\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$) whose columns are the eigenvectors, and $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ is a diagonal matrix of eigenvalues.

Proof that eigenvalues of a real symmetric matrix are real. Let $\mathbf{M} = \mathbf{M}^T$ and suppose $\mathbf{M}\mathbf{v} = \lambda\mathbf{v}$ for some possibly complex eigenvalue λ and eigenvector $\mathbf{v}$. We will show that λ must be real.

Take the complex conjugate transpose (denoted by *) of both sides of $\mathbf{M}\mathbf{v} = \lambda\mathbf{v}$:

$$\mathbf{v}^*\mathbf{M}^T = \bar{\lambda}\,\mathbf{v}^* \tag{1.36}$$

Since $\mathbf{M} = \mathbf{M}^T$ (and $\mathbf{M}$ is real, so $\mathbf{M}^* = \mathbf{M}^T = \mathbf{M}$):

$$\mathbf{v}^*\mathbf{M} = \bar{\lambda}\,\mathbf{v}^* \tag{1.37}$$

Now multiply the original equation on the left by $\mathbf{v}^*$:

$$\mathbf{v}^*\mathbf{M}\mathbf{v} = \lambda\,\mathbf{v}^*\mathbf{v} \tag{1.38}$$

And multiply the conjugated equation on the right by $\mathbf{v}$:

$$\mathbf{v}^*\mathbf{M}\mathbf{v} = \bar{\lambda}\,\mathbf{v}^*\mathbf{v} \tag{1.39}$$

Equating the right-hand sides:

$$\lambda\,\mathbf{v}^*\mathbf{v} = \bar{\lambda}\,\mathbf{v}^*\mathbf{v} \tag{1.40}$$

Since $\mathbf{v} \neq \mathbf{0}$, we have $\mathbf{v}^*\mathbf{v} = \sum_i |v_i|^2 > 0$. Dividing both sides by $\mathbf{v}^*\mathbf{v}$ yields $\lambda = \bar{\lambda}$, which is true if and only if λ is real. □ □

Proof that eigenvectors for distinct eigenvalues are orthogonal. Let $\mathbf{M}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$ and $\mathbf{M}\mathbf{v}_2 = \lambda_2\mathbf{v}_2$ with $\lambda_1 \neq \lambda_2$. Compute:

$$\lambda_1(\mathbf{v}_1^T\mathbf{v}_2) = (\mathbf{M}\mathbf{v}_1)^T\mathbf{v}_2 = \mathbf{v}_1^T\mathbf{M}^T\mathbf{v}_2 = \mathbf{v}_1^T\mathbf{M}\mathbf{v}_2 = \mathbf{v}_1^T(\lambda_2\mathbf{v}_2) = \lambda_2(\mathbf{v}_1^T\mathbf{v}_2) \tag{1.41}$$

Therefore $(\lambda_1 - \lambda_2)(\mathbf{v}_1^T\mathbf{v}_2) = 0$. Since $\lambda_1 \neq \lambda_2$, we must have $\mathbf{v}_1^T\mathbf{v}_2 = 0$. The eigenvectors are orthogonal. □ □

The spectral decomposition $\mathbf{M} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$ has a beautiful geometric interpretation: the transformation defined by $\mathbf{M}$ can be decomposed into three steps: (1) rotate coordinates to align with the eigenvector axes ($\mathbf{Q}^T$), (2) independently scale each axis by its eigenvalue ($\mathbf{\Lambda}$), (3) rotate back ($\mathbf{Q}$). This is a "rotation-stretch-rotation" decomposition entirely analogous to the SVD, which we discuss next.

1.4.4 Quadratic Forms and Positive Definiteness

In optimization and energy mechanics, we frequently encounter *quadratic forms*—scalar-valued functions that are quadratic in a vector argument:

Definition 1.11. Quadratic Form

Given a symmetric matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$, the associated *quadratic form* is the scalar function $q : \mathbb{R}^n \to \mathbb{R}$ defined by:

$$q(\mathbf{x}) = \mathbf{x}^T \mathbf{M} \mathbf{x} = \sum_{i=1}^{n} \sum_{j=1}^{n} M_{ij}\, x_i\, x_j \tag{1.42}$$

Quadratic forms appear throughout this text series:

- **Kinetic energy:** $T = \frac{1}{2}\dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}$, where $\mathbf{M}(\mathbf{q})$ is the mass matrix (Chapter **??**).
- **Optimal control cost:** $J = \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}$, where $\mathbf{Q}$ penalizes state deviations and $\mathbf{R}$ penalizes control effort.
- **Lyapunov functions:** $V(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{x}$, used to prove stability (Volume I).
- **Contraction metrics:** $\delta\mathbf{x}^T \mathbf{M}(x)\, \delta\mathbf{x}$, measuring infinitesimal distances on manifolds (Volume I).

Definition 1.12. Positive Definiteness

A symmetric matrix $\mathbf{M}$ is called:

- **Positive definite** ($\mathbf{M} \succ 0$) if $\mathbf{x}^T \mathbf{M} \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$.
- **Positive semi-definite** ($\mathbf{M} \succeq 0$) if $\mathbf{x}^T \mathbf{M} \mathbf{x} \geq 0$ for all $\mathbf{x}$.
- **Negative definite** ($\mathbf{M} \prec 0$) if $\mathbf{x}^T \mathbf{M} \mathbf{x} < 0$ for all $\mathbf{x} \neq \mathbf{0}$.
- **Indefinite** if $\mathbf{x}^T \mathbf{M} \mathbf{x}$ takes both positive and negative values.

Proposition 1.2. *A symmetric matrix $\mathbf{M}$ is positive definite if and only if all of its eigenvalues are strictly positive.*

Proof. By the Spectral Theorem (Theorem 1.3), $\mathbf{M} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$ where $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ and $\mathbf{Q}$ is orthogonal. For any nonzero $\mathbf{x}$, define $\mathbf{y} = \mathbf{Q}^T \mathbf{x}$. Since $\mathbf{Q}$ is invertible and $\mathbf{x} \neq \mathbf{0}$,

we have $\mathbf{y} \neq \mathbf{0}$. Then:

$$\mathbf{x}^T\mathbf{M}\mathbf{x} = \mathbf{x}^T\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T\mathbf{x} = \mathbf{y}^T\mathbf{\Lambda}\mathbf{y} = \sum_{i=1}^{n} \lambda_i y_i^2 \tag{1.43}$$

($\Rightarrow$) If $\mathbf{M} \succ 0$, then for each eigenvector $\mathbf{v}_i$ (which is nonzero): $\mathbf{v}_i^T\mathbf{M}\mathbf{v}_i = \lambda_i \|\mathbf{v}_i\|^2 > 0$, so $\lambda_i > 0$.

($\Leftarrow$) If all $\lambda_i > 0$ and $\mathbf{y} \neq \mathbf{0}$, then at least one $y_i \neq 0$, so $\sum_i \lambda_i y_i^2 > 0$. □ □

In Plain Language 1.5. Why Positive Definiteness Matters for Control

In control theory, if you can construct a function $V(\mathbf{x}) = \mathbf{x}^T\mathbf{P}\mathbf{x}$ with $\mathbf{P} \succ 0$ such that V decreases along every trajectory of your system, you have mathematically *proven* that the system is stable. The positive definiteness of $\mathbf{P}$ guarantees that V has a strict minimum at the origin—it forms a "bowl" in state space. If the system's energy (V) always decreases and the bowl has a definite bottom, the system must converge to rest. This is the essence of **Lyapunov stability theory**, the single most important stability analysis tool in nonlinear control (developed in Volume I).

The Cholesky Decomposition

A practical consequence of positive definiteness: any positive definite matrix $\mathbf{M}$ can be uniquely factored as:

$$\mathbf{M} = \mathbf{L}\mathbf{L}^T \tag{1.44}$$

where $\mathbf{L}$ is a lower triangular matrix with strictly positive diagonal entries. This is the **Cholesky decomposition**. It is numerically efficient (roughly half the cost of a general LU factorization) and is the standard method for solving linear systems involving positive definite matrices in physics engines. When MuJoCo inverts the mass matrix $\mathbf{M}(\mathbf{q})$ to compute forward dynamics, it uses Cholesky decomposition.

The Cholesky factor $\mathbf{L}$ also provides a constructive test for positive definiteness: if the factorization completes without encountering a zero or negative diagonal element, the matrix is positive definite. If the algorithm fails, the matrix is not positive definite.

1.5 The Singular Value Decomposition (SVD)

What if a matrix is not square? What if it does not have a full set of eigenvectors? The **Singular Value Decomposition** (SVD) is the ultimate, universal factorization of *any* matrix $\mathbf{M} \in \mathbb{R}^{n\times m}$, regardless of its shape or rank.

Theorem 1.4. The Singular Value Decomposition

Every matrix $\mathbf{M} \in \mathbb{R}^{n\times m}$ can be factored as:

$$\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{1.45}$$

where:

- $\mathbf{U} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix ($\mathbf{U}^T\mathbf{U} = \mathbf{I}$). Its columns are called the *left singular vectors*.
- $\mathbf{\Sigma} \in \mathbb{R}^{n \times m}$ is a diagonal matrix with non-negative entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(n,m)} \geq 0$ on the diagonal. These are the *singular values*.
- $\mathbf{V} \in \mathbb{R}^{m \times m}$ is an orthogonal matrix. Its columns are the *right singular vectors*.

1.5.1 Derivation of the SVD from the Eigendecomposition

The SVD can be derived from the spectral theorem applied to related symmetric matrices. Here is the construction:

Step 1. Form the symmetric matrix $\mathbf{M}^T\mathbf{M} \in \mathbb{R}^{m \times m}$. This matrix is always positive semi-definite because for any $\mathbf{x}$:

$$\mathbf{x}^T(\mathbf{M}^T\mathbf{M})\mathbf{x} = (\mathbf{M}\mathbf{x})^T(\mathbf{M}\mathbf{x}) = \|\mathbf{M}\mathbf{x}\|^2 \geq 0 \tag{1.46}$$

Step 2. By the Spectral Theorem, $\mathbf{M}^T\mathbf{M}$ has an eigendecomposition:

$$\mathbf{M}^T\mathbf{M} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T \tag{1.47}$$

where $\mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_m)$ with $\lambda_i \geq 0$ (since $\mathbf{M}^T\mathbf{M}$ is positive semi-definite). Define the singular values as $\sigma_i = \sqrt{\lambda_i}$.

Step 3. For each nonzero singular value σ_i, define the corresponding left singular vector:

$$\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{M}\mathbf{v}_i \tag{1.48}$$

These vectors are orthonormal (a straightforward verification using $\mathbf{M}^T\mathbf{M}\mathbf{v}_i = \sigma_i^2\mathbf{v}_i$).

Step 4. Extend $\{\mathbf{u}_1, \ldots, \mathbf{u}_r\}$ to a complete orthonormal basis for $\mathbb{R}^n$ (where $r = \text{rank}(\mathbf{M})$). Assemble these into the columns of $\mathbf{U}$. Then $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ follows by construction.

1.5.2 Geometric Interpretation of the SVD

The SVD reveals that *any* linear transformation, no matter how complex, can be decomposed into exactly three simple geometric operations:

1. **Rotate the input space** (apply $\mathbf{V}^T$): align the input with the principal directions.
2. **Scale each axis independently** (apply $\mathbf{\Sigma}$): stretch or shrink along each principal direction.
3. **Rotate the output space** (apply $\mathbf{U}$): reorient the result in the output space.

This geometric decomposition transforms a unit hypersphere in the input space into a hyperellipsoid in the output space. The singular values are the semi-axis lengths of this ellipsoid, and the left singular vectors point along the ellipsoid's principal axes.

1.5.3 The Condition Number

The **condition number** of a matrix $\mathbf{M}$ is the ratio of its largest to its smallest singular value:

$$\kappa(\mathbf{M}) = \frac{\sigma_{\max}}{\sigma_{\min}} \tag{1.49}$$

If $\kappa(\mathbf{M})$ is large (say, 10^6 or more), the matrix is *ill-conditioned*: small perturbations in the input can cause enormous changes in the output. In robotics, the condition number of the Jacobian quantifies how close the robot is to a singularity. As $\sigma_{\min} \to 0$, the condition number $\to \infty$, and the numerical solution of $\mathbf{J}\dot{\mathbf{q}} = \mathbf{v}$ becomes catastrophically unreliable.

1.5.4 Truncated SVD and Low-Rank Approximation

One of the most powerful applications of the SVD is data compression. Given a matrix $\mathbf{M}$ with singular values $\sigma_1 \geq \sigma_2 \geq \cdots$, the best rank-$k$ approximation (in the Frobenius norm) is:

$$\mathbf{M}_k = \sum_{i=1}^{k} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^T \tag{1.50}$$

This result (the Eckart-Young-Mirsky theorem) underpins dimensionality reduction techniques like Principal Component Analysis (PCA), which identifies the dominant modes of variation in high-dimensional data such as motion capture recordings of golf swings.

Example 1.5. The SVD of the Jacobian (Manipulability Ellipsoid)

In robotics, the Jacobian matrix $\mathbf{J}(\mathbf{q})$ maps joint velocities to end-effector Cartesian velocity: $\mathbf{v}_{\text{hand}} = \mathbf{J}\dot{\mathbf{q}}$. For a redundant manipulator with 7 joints moving in 3D space, $\mathbf{J} \in \mathbb{R}^{3\times 7}$. Taking the SVD $\mathbf{J} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$:

- $\mathbf{U} \in \mathbb{R}^{3\times 3}$: The columns define the physical 3D directions the hand can move.
- $\mathbf{\Sigma} \in \mathbb{R}^{3\times 7}$: The singular values $(\sigma_1, \sigma_2, \sigma_3)$ are the semi-axis lengths of the **Velocity Manipulability Ellipsoid**. They quantify how fast the robot can move in each $\mathbf{U}$-direction. If the arm is stretched perfectly straight, one singular value drops to $\sigma_k = 0$: the ellipsoid collapses to a pancake, and the robot loses a degree of freedom.
- $\mathbf{V}^T \in \mathbb{R}^{7\times 7}$: The columns define the joint velocity combinations required to produce motion along each principal direction.

Conversely, the **Force Manipulability Ellipsoid** has axes proportional to $1/\sigma_i$. Where the robot is fastest (σ_i large), it is mechanically weakest. Where it is slowest (near singularities, σ_i small), it can exert enormous force—a consequence of the conservation of mechanical power.

1.6 Dyads, Dyadics, and Tensor Products

The matrix decompositions we have studied so far—eigendecomposition, SVD—break a matrix into pieces that reveal its geometric action. In this section we meet a complementary idea: building matrices up from the simplest possible pieces, called *dyads*. This viewpoint is not merely algebraic; it is the language that mechanics has used for over a century to describe inertia, stress, and stiffness without ever choosing a coordinate frame [**?**].

1.6.1 What Is a Dyad?

Definition 1.13. Dyad (Outer Product)

Given two vectors $\mathbf{a} \in \mathbb{R}^m$ and $\mathbf{b} \in \mathbb{R}^n$, the **dyad** (or **outer product**) is the $m \times n$ matrix

$$\mathbf{D} \;=\; \mathbf{a}\,\mathbf{b}^T.$$

This matrix has rank at most one: every column is a scalar multiple of $\mathbf{a}$, and every row is a scalar multiple of $\mathbf{b}^T$.

A dyad encodes *two* directions simultaneously. When it acts on a vector $\mathbf{v}$,

$$\mathbf{D}\,\mathbf{v} \;=\; (\mathbf{a}\,\mathbf{b}^T)\mathbf{v} \;=\; \mathbf{a}\,\langle \mathbf{b},\, \mathbf{v}\rangle\,,$$

it first *senses* how much $\mathbf{v}$ aligns with $\mathbf{b}$ (via the inner product $\langle \mathbf{b}, \mathbf{v}\rangle$) and then *responds* along the direction $\mathbf{a}$, scaled by that alignment.

In Plain Language 1.6. Dyads: Direction of Action and Direction of Sensitivity

Imagine a spring that is stiff in one direction but compliant in another—say, a car suspension that resists vertical bumps but allows lateral flex. The relationship "sense displacement in direction $\mathbf{b}$, produce force along direction $\mathbf{a}$" is exactly a dyad $\mathbf{a}\,\mathbf{b}^T$. Think of it as a *cause-and-effect pair*: the vector $\mathbf{b}$ is the "listening direction" (what the spring feels) and $\mathbf{a}$ is the "acting direction" (how the spring pushes back). A single dyad can only do one such pairing, which is why its matrix has rank one—it has only one independent cause-to-effect channel.

Example 1.6. A Rank-1 Force Model

Let $\mathbf{a} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$ (vertical) and $\mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ (horizontal). Then

$$\mathbf{D} = \mathbf{a}\,\mathbf{b}^T = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Applied to a displacement $\mathbf{v} = \begin{bmatrix} 3 \\ 5 \\ 2 \end{bmatrix}$:

$$\mathbf{D}\,\mathbf{v} = \begin{bmatrix} 0 \\ 3 \\ 0 \end{bmatrix}.$$

The dyad extracts the horizontal component (3) and produces a purely vertical response—exactly the one-channel behaviour described above.

1.6.2 From Dyads to Dyadics

A single dyad is rank one, but real physical relationships—stiffness, inertia, stress—have full rank. We get there by *summing* dyads.

Definition 1.14. Dyadic (Tensor Product Expansion)

A **dyadic** is a finite sum of dyads:

$$\mathbf{T} \;=\; \sum_{k=1}^{r} \mathbf{a}_k\,\mathbf{b}_k^T.$$

Every $m \times n$ matrix can be written in this form with $r \le \min(m, n)$ terms.

This is not a new fact—it is the column–row factorisation you already know from the SVD. If $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$, then

$$\mathbf{A} \;=\; \sum_{k=1}^{r} \sigma_k\,\mathbf{u}_k\,\mathbf{v}_k^T,$$

which is a dyadic with r terms (the rank of $\mathbf{A}$). The SVD merely chooses the *orthogonal* dyad decomposition; infinitely many other decompositions exist.

The connection to abstract algebra: the space $\mathbb{R}^m \otimes \mathbb{R}^n$ (the **tensor product**) is isomorphic to $\mathbb{R}^{m\times n}$. A dyad $\mathbf{a}\,\mathbf{b}^T$ corresponds to the elementary tensor $\mathbf{a}\otimes\mathbf{b}$, and a general element of the tensor product space is a sum of such elementary tensors—a dyadic.

1.6.3 Physical Meaning in Mechanics

The power of the dyadic viewpoint is that it names the two "slots" of a matrix: input direction and output direction. Three central objects in mechanics are dyadics.

The inertia tensor. The rotational inertia of a rigid body maps angular velocity to angular momentum:

$$\mathbf{L} \;=\; \mathbf{I}\,\boldsymbol{\omega}. \tag{1.51}$$

Here $\boldsymbol{\omega}$ is the "input" (the axis and rate of spin) and $\mathbf{L}$ is the "output" (the direction and magnitude of angular momentum). Unless the body has special symmetry, $\mathbf{L}$ and $\boldsymbol{\omega}$ point in different directions; the inertia tensor is the dyadic that encodes this directional coupling.

The stress tensor. The Cauchy stress tensor $\boldsymbol{\sigma}$ maps a surface-normal direction $\hat{\mathbf{n}}$ to the traction (force per unit area) $\mathbf{t}$ acting on that surface:

$$\mathbf{t} \;=\; \boldsymbol{\sigma}\,\hat{\mathbf{n}}.$$

Input: the orientation of a small area element. Output: the force that the material on one side exerts on the other.

The stiffness matrix. In structural mechanics, the stiffness matrix $\mathbf{K}$ maps a displacement vector $\mathbf{u}$ to a force vector $\mathbf{f}$: $\mathbf{f} = \mathbf{K}\,\mathbf{u}$. For a truss with N members, each member contributes a rank-1 dyad (stiffness along its axis), and the global stiffness matrix is the sum:

$$\mathbf{K} \;=\; \sum_{e=1}^{N} k_e\,\hat{\mathbf{n}}_e\,\hat{\mathbf{n}}_e^{T},$$

a dyadic built from the member directions $\hat{\mathbf{n}}_e$ and axial stiffnesses k_e.

In Plain Language 1.7. Why Engineers Say "Tensor"

When a mechanical engineer refers to the "inertia tensor" or "stress tensor," they almost always mean a 3×3 symmetric matrix that represents a dyadic. The word *tensor* signals that the object transforms in a specific, predictable way under changes of coordinate frame. It is not just a bag of nine numbers; it is a physical machine that takes one vector in and produces another vector out, regardless of how you choose your axes.

1.6.4 The Coordinate-Free Perspective

A dyadic is a geometric object that lives in the space of linear maps $\mathbb{R}^n \to \mathbb{R}^m$. Choosing a basis gives it a matrix representation, but the underlying map does not depend on that choice.

Example 1.7. Inertia Tensor of a Point Mass

A point mass m at position $\mathbf{r}$ from the centre of mass has inertia tensor

$$\mathbf{I} \;=\; m\big(\|\mathbf{r}\|^2\,\mathbf{1} - \mathbf{r}\,\mathbf{r}^{T}\big), \tag{1.52}$$

where $\mathbf{1}$ is the 3×3 identity. Notice that no basis vectors appear—$\mathbf{r}\,\mathbf{r}^T$ is a dyad formed from the position vector with itself, and $\mathbf{1}$ is the identity map. This expression is *coordinate-free*: it is valid in every frame simultaneously.

When we change from a body-fixed frame $\{B\}$ to a world frame $\{W\}$ via a rotation $\mathbf{R} \in \mathrm{SO}(3)$, the matrix representation of the inertia tensor transforms by the **congruence transformation**:

$$\mathbf{I}^{W} \;=\; \mathbf{R}\,\mathbf{I}^{B}\,\mathbf{R}^{T}. \tag{1.53}$$

The congruence form $\mathbf{R}\,(\cdot)\,\mathbf{R}^T$ rotates both the input slot and the output slot of the dyadic. It preserves symmetry, positive-definiteness, and—for rotations—eigenvalues (the principal moments of inertia).

1.6.5 Connection to the Rest of This Book

The dyadic point of view recurs throughout our study of rigid-body mechanics:

- **Screw axes and wrenches (Chapter 5).** A wrench combines a force and a moment into a single six-dimensional object. The mapping from a twist (generalised velocity) to a wrench (generalised force) is a 6×6 dyadic—the *spatial inertia*.
- **Spatial algebra (Chapter 8).** The 6×6 spatial inertia matrix is a dyadic on the space of twists. Its coordinate-free formulation uses the same $m(\|\mathbf{r}\|^2\,\mathbf{1} - \mathbf{r}\,\mathbf{r}^T)$ building block we saw in Example 1.7, extended to six dimensions.
- **The mass matrix (Chapter 11).** The joint-space mass matrix $\mathbf{M}(\mathbf{q})$ that appears in the manipulator equation $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{c} = \boldsymbol{\tau}$ is a configuration-dependent dyadic on joint-velocity space. Each link contributes a term built from Jacobians and spatial inertias—a sum of dyads, assembled exactly as we sum member stiffnesses in a truss.
- **The articulated-body algorithm (Chapter 10).** Featherstone's algorithm propagates *articulated-body inertias* through a kinematic tree. These are 6×6 dyadics that account for the coupling between a link's motion and the reactions of all its descendants.

Understanding that all these objects are dyadics—linear maps built from sums of outer products—provides a unifying thread: the same algebraic structure appears whether we are analysing a truss, spinning a gyroscope, or computing the forward dynamics of a humanoid robot.

1.7 Matrix Norms and Conditioning

Having decomposed matrices via the eigendecomposition and SVD, we now turn to a fundamental practical question: *how large is a matrix, and how sensitive is a linear system to perturbations?* Matrix norms quantify the first question; the *condition number* answers the second.

Definition 1.15. Frobenius Norm $\|\mathbf{A}\|_F$

The **Frobenius norm** of a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is

$$\|\mathbf{A}\|_F \;=\; \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2} \;=\; \sqrt{\operatorname{tr}\!\left(\mathbf{A}^T\mathbf{A}\right)} \;=\; \sqrt{\sigma_1^2 + \sigma_2^2 + \cdots + \sigma_{\min(m,n)}^2}. \tag{1.54}$$

The last equality follows directly from the SVD. The Frobenius norm treats the matrix as a long vector and computes its Euclidean length.

Definition 1.16. Spectral Norm $\|\mathbf{A}\|_2$

The **spectral norm** (also called the *operator 2-norm* or *induced 2-norm*) is

$$\|\mathbf{A}\|_2 \;=\; \max_{\|\mathbf{x}\|_2=1} \|\mathbf{A}\mathbf{x}\|_2 \;=\; \sigma_{\max}(\mathbf{A}). \tag{1.55}$$

This is the maximum factor by which $\mathbf{A}$ can stretch a unit vector—exactly the length of the longest semi-axis of the image ellipsoid from the SVD's geometric interpretation (Section **??**).

Definition 1.17. Condition Number $\kappa(\mathbf{A})$

For a matrix $\mathbf{A}$ with $\sigma_{\min} > 0$, the **condition number** with respect to the 2-norm is

$$\kappa(\mathbf{A}) \;=\; \frac{\sigma_{\max}}{\sigma_{\min}}. \tag{1.56}$$

When $\sigma_{\min} = 0$ (i.e., $\mathbf{A}$ is singular), we define $\kappa(\mathbf{A}) = \infty$. A well-conditioned matrix has $\kappa \approx 1$; an ill-conditioned matrix has $\kappa \gg 1$.

Physical interpretation: Jacobian conditioning near singularities. Consider the manipulator Jacobian $\mathbf{J}(\mathbf{q})$ from the SVD example (Example 1.5). When the robot approaches a kinematic singularity, one or more singular values $\sigma_i \to 0$ while others remain bounded. The condition number $\kappa(\mathbf{J})$ therefore grows without bound. Because the solution to $\dot{\mathbf{q}} = \mathbf{J}^{-1}\mathbf{v}$ amplifies input errors by a factor proportional to κ, even tiny errors in the desired Cartesian velocity $\mathbf{v}$ produce enormous joint velocities. This is why real robots exhibit jerky, unstable motion near singular configurations [**?**].

In Plain Language 1.8. Why Does Your Robot Arm Shake Near Singularities?

Why does your robot arm shake near singularities? Because the condition number of the Jacobian explodes. When $\kappa(\mathbf{J})$ is huge, the inverse mapping from Cartesian velocity to joint velocity amplifies every small measurement error, rounding error, and servo lag into wild joint commands. Monitoring $\kappa(\mathbf{J})$ in real time is standard practice in industrial controllers: when the condition number exceeds a threshold, the controller switches to a damped pseudo-inverse to keep the motion smooth.

1.8 Matrix Decompositions for Computation

The eigendecomposition and SVD reveal geometric structure. This section surveys three additional decompositions whose primary purpose is *computational efficiency* when solving the linear systems that arise throughout dynamics and control.

1.8.1 LU Decomposition

Any invertible matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ can (with row permutations) be factored as

$$\mathbf{PA} = \mathbf{LU}, \tag{1.57}$$

where $\mathbf{P}$ is a permutation matrix, $\mathbf{L}$ is unit lower triangular, and $\mathbf{U}$ is upper triangular. Solving $\mathbf{Ax} = \mathbf{b}$ then reduces to two triangular solves (forward and back substitution), each costing $O(n^2)$ after the $O(n^3)$ factorization. When multiple right-hand sides share the same $\mathbf{A}$, LU is the workhorse: factor once, solve many times.

1.8.2 QR Decomposition

For any $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m \geq n$,

$$\mathbf{A} = \mathbf{QR}, \tag{1.58}$$

where $\mathbf{Q} \in \mathbb{R}^{m\times m}$ is orthogonal and $\mathbf{R} \in \mathbb{R}^{m\times n}$ is upper triangular. QR is the preferred method for solving least-squares problems $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2$ because, unlike the normal equations $\mathbf{A}^T\mathbf{Ax} = \mathbf{A}^T\mathbf{b}$, it avoids squaring the condition number ($\kappa(\mathbf{A}^T\mathbf{A}) = \kappa(\mathbf{A})^2$) and is therefore far more numerically stable.

1.8.3 Cholesky Decomposition

If $\mathbf{M} \in \mathbb{R}^{n\times n}$ is symmetric positive definite (SPD), then there exists a unique lower-triangular matrix $\mathbf{L}$ with positive diagonal entries such that

$$\mathbf{M} = \mathbf{LL}^T. \tag{1.59}$$

Cholesky costs roughly half the flops of LU and is always numerically stable for SPD matrices—no pivoting is needed.

Mathematical Principle 1.1. Cholesky and the Mass Matrix

The generalized mass matrix $\mathbf{M}(\mathbf{q})$ that appears in the manipulator equation of motion (Chapter 11) is always symmetric positive definite. Therefore the Cholesky decomposition is the preferred solver for the forward-dynamics equation $\mathbf{M}\ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{C}\dot{\mathbf{q}} - \mathbf{g}$ [?].

1.9 Worked Example: Eigenvalue and SVD Analysis in Python

Let us bring together the theory developed in this chapter with a concrete computational example. The following Python code constructs a 3×3 symmetric matrix (representing a simplified 3D inertia tensor), computes its eigendecomposition and SVD, and verifies the key properties we have proven.

```
import numpy as np

# ---- Define a symmetric positive definite matrix ----
# This could represent a simplified 3D inertia tensor
M = np.array([
```

```
    [5.0, 1.0, 0.5],
    [1.0, 4.0, 0.3],
    [0.5, 0.3, 3.0]
])

# Verify symmetry
assert np.allclose(M, M.T), "Matrix is not symmetric!"

# ---- Eigendecomposition ----
eigenvalues, eigenvectors = np.linalg.eigh(M)  # eigh for symmetric matrices
print("Eigenvalues:", np.round(eigenvalues, 4))
print("Eigenvectors (columns):\n", np.round(eigenvectors, 4))

# Verify: all eigenvalues positive => positive definite
assert all(eigenvalues > 0), "Matrix is not positive definite!"
print("Matrix is POSITIVE DEFINITE (all eigenvalues > 0)")

# Verify: eigenvectors are orthogonal
QTQ = eigenvectors.T @ eigenvectors
print("Q^T Q (should be identity):\n", np.round(QTQ, 10))

# Verify spectral decomposition: M = Q * Lambda * Q^T
M_reconstructed = eigenvectors @ np.diag(eigenvalues) @ eigenvectors.T
print("Reconstruction error:", np.linalg.norm(M - M_reconstructed))

# ---- Singular Value Decomposition ----
U, sigma, VT = np.linalg.svd(M)
print("\nSingular values:", np.round(sigma, 4))
print("Eigenvalues (sorted descending):", np.round(sorted(eigenvalues,
    reverse=True), 4))
print("For symmetric PD matrices, singular values = eigenvalues (sorted)")

# ---- Condition Number ----
kappa = sigma[0] / sigma[-1]
print(f"\nCondition number: {kappa:.4f}")
print("(Close to 1 means well-conditioned; >>1 means ill-conditioned)")

# ---- Cholesky Decomposition ----
L = np.linalg.cholesky(M)
print("\nCholesky factor L:\n", np.round(L, 4))
print("Verify L @ L^T = M, error:", np.linalg.norm(M - L @ L.T))
```

Listing 1.1: Eigendecomposition and SVD of a Symmetric Matrix

1.10 Chapter Summary

Mathematical Principle 1.2. Key Takeaways from Chapter 1

1. A **vector space** is defined by eight axioms guaranteeing closure, commutativity, and compatibility of addition and scalar multiplication. $\mathbb{R}^n$ is the prototypical example.

2. The **rank** of a matrix counts independent columns; the **null space** captures the

"lost" dimensions. The **Rank-Nullity Theorem** connects them: rank+nullity = number of columns.

3. The **dot product** measures alignment between vectors and defines orthogonality. The **cross product** in $\mathbb{R}^3$ produces a perpendicular vector and can be written as a *skew-symmetric matrix* multiplication—a representation central to Lie Algebra ($\mathfrak{so}(3)$).

4. **Eigenvalues** of a dynamics matrix dictate stability; eigenvalues of an inertia tensor identify principal axes. The **Spectral Theorem** guarantees that symmetric matrices have real eigenvalues and orthogonal eigenvectors.

5. **Positive definiteness** ($\mathbf{M} \succ 0$) guarantees a quadratic form has a strict minimum—the cornerstone of Lyapunov stability proofs and energy-based control design.

6. The **SVD** decomposes any matrix into rotation-stretch-rotation. It reveals the manipulability ellipsoid of a robot and quantifies proximity to singularities via the condition number.

1.11 Further Reading

The material in this chapter draws on a long tradition of textbooks at the intersection of linear algebra and applied mathematics. For a comprehensive undergraduate treatment that emphasizes geometric intuition alongside computational technique, Strang [**?**] remains the standard reference; his "four fundamental subspaces" perspective directly informs the rank-nullity discussion presented here.

In the robotics context, Chapter 2 of Murray, Li, and Sastry [**?**] develops the specific linear-algebraic tools—rotation matrices, skew-symmetric operators, and the Lie bracket—that recur throughout the rest of this textbook. Readers who wish to pursue the numerical aspects of linear algebra, particularly iterative solvers and the role of condition numbers in optimization, will find Nocedal and Wright [**?**] an invaluable companion.

The dyadic and tensor-product viewpoint introduced in Section 1.6 connects directly to continuum mechanics, where stress and strain are second-order tensors built from exactly the same outer-product construction. Students interested in these connections should consult any graduate text on continuum mechanics or tensor analysis.

Finally, the eigenvalue structures and matrix exponentials that close this chapter are the first hints of *Lie theory*. Hall [**?**] provides a mathematically rigorous bridge from linear algebra to Lie groups and Lie algebras, foreshadowing the rotation groups (SO(3)(3)) and rigid-body transformation groups (SE(3)(3)) that appear in Chapters 4 and 5.

1.12 Exercises

Computational Exercises

1. **(Rank and Null Space)** Given the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

compute the rank, determinant, and a basis for the null space. Verify the Rank-Nullity Theorem. *Hint: row reduce to echelon form.*

2. **(Eigenvalue Computation)** For the matrix $\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$, compute the eigenvalues and eigenvectors by hand. Verify that $\mathbf{A}\mathbf{v}_i = \lambda_i \mathbf{v}_i$ for each pair.

3. **(SVD by Hand)** Compute the SVD of $\mathbf{M} = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$. What are the singular values, and how do they relate to the eigenvalues?

4. **(Cholesky)** Verify that $\mathbf{M} = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$ is positive definite by: (a) checking all eigenvalues are positive, (b) computing the Cholesky factorization $\mathbf{L}\mathbf{L}^T$.

5. **(Python)** Write a Python script that generates a random 5×5 symmetric matrix $\mathbf{M} = \mathbf{A}^T\mathbf{A} + \epsilon\mathbf{I}$ (guaranteeing positive definiteness), computes its eigendecomposition, SVD, and Cholesky factorization, and verifies all three decompositions reproduce the original matrix within numerical tolerance.

Conceptual Exercises

6. **(Vector Spaces)** Is the set of all 2×2 symmetric matrices a vector space under standard matrix addition and scalar multiplication? Prove or disprove by checking the axioms. What is its dimension?

7. **(Physical Interpretation)** A robot Jacobian $\mathbf{J} \in \mathbb{R}^{3\times5}$ has singular values $\sigma_1 = 2.0$, $\sigma_2 = 0.8$, $\sigma_3 = 0.01$. Describe the shape of the velocity manipulability ellipsoid. Is the robot near a singularity? What does the condition number tell you about the reliability of inverse kinematics computations?

8. **(Positive Definiteness in Control)** Explain in your own words why the mass matrix $\mathbf{M}(\mathbf{q})$ of any physical mechanical system must be positive definite. What would a zero eigenvalue of $\mathbf{M}(\mathbf{q})$ physically mean?

9. **(Skew-Symmetric Matrices)** Prove that for any skew-symmetric matrix $\mathbf{S}$ ($\mathbf{S}^T = -\mathbf{S}$), the matrix $e^{\mathbf{S}} = \mathbf{I} + \mathbf{S} + \frac{1}{2!}\mathbf{S}^2 + \cdots$ is an orthogonal matrix (i.e., $(e^{\mathbf{S}})^T e^{\mathbf{S}} = \mathbf{I}$). *Hint: use the fact that* $(e^{\mathbf{S}})^T = e^{\mathbf{S}^T} = e^{-\mathbf{S}}$. This result is the mathematical foundation of Chapter **??**.

Proof-Based Exercises

10. **(Rank-Nullity Application)** Prove that if $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m < n$, then $\mathbf{A}$ must have a nontrivial null space (i.e., $\mathcal{N}(\mathbf{A}) \neq \{\mathbf{0}\}$). Interpret this result for an underactuated robot with more joints than task-space dimensions.

11. **(Trace and Eigenvalues)** Prove that the trace of a matrix (sum of diagonal elements) equals the sum of its eigenvalues: $\text{tr}(\mathbf{M}) = \sum_{i=1}^{n} \lambda_i$. *Hint: use the fact that similar matrices have the same trace.*

12. **(Determinant and Eigenvalues)** Prove that the determinant of a matrix equals the product of its eigenvalues: $\det(\mathbf{M}) = \prod_{i=1}^{n} \lambda_i$. *Hint: evaluate the characteristic polynomial at $\lambda = 0$.*

13. **(Uniqueness of SVD)** The singular values of a matrix are unique, but the singular vectors are not (when singular values are repeated). Give an example of a 2×2 matrix with a repeated singular value, and show that the corresponding singular vectors can be chosen as any orthonormal pair spanning a two-dimensional subspace.

14. **(Schur Complement and Positive Definiteness)** Let $\mathbf{M} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^T & \mathbf{C} \end{bmatrix}$ be a symmetric block matrix with $\mathbf{A} \succ 0$. Prove that $\mathbf{M} \succ 0$ if and only if $\mathbf{C} - \mathbf{B}^T\mathbf{A}^{-1}\mathbf{B} \succ 0$ (the *Schur complement* condition). This result is used extensively in the Articulated Body Algorithm (Chapter **??**).

Chapter 2

The Transition to State Space

You cannot control a machine by staring at the machine. You control it by staring at the mathematical space that completely defines it.

2.1 Introduction: The Physics Abstraction

In Plain Language 2.1. Context & Use Case: Why We Banish 3D Physics

The Math You Will See: We will build the **State Vector** ($\mathbf{x}$), a vertical stack of numbers containing positions ($\mathbf{q}$) and velocities ($\dot{\mathbf{q}}$). We will formulate the universal control equation $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, t)$, and its linear cousin $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$.
Why We Need It: When a robotic arm swings a bat, the shoulder, elbow, and wrist all move in weird, overlapping arcs in 3D physics space (X, Y, Z). This is impossibly hard to calculate directly. Instead, we map the entire robot into an abstract mathematical realm called "State Space". In State Space, the complex multi-joint swinging robot is represented by a *single, infinitesimally small dot* moving along a single curved line. This abstraction makes solving incredibly complex control problems possible.
All Variables Defined: $\mathbf{x}$ is the state, $\mathbf{u}$ is the control input (e.g., motor torque), and f is the system dynamics mapping. See the **Nomenclature** at the start of the book for a full reference.

When a layman pictures a robotic arm throwing a ball or a drone correcting its pitch in a crosswind, they visualize physical bodies acting in 3-dimensional Euclidean space.

But control theorists and kinematic engineers almost never look at the physical space. They work in a wholly abstract, higher-dimensional mathematical realm known as **State Space**. State space is arguably the most fundamental translation required in control theory. If you do not understand state space, nothing else in the subsequent volumes of *The Geometry of Motion* will make sense.

2.2 Historical Context: The Kalman Revolution

In Plain Language 2.2. Historical Context: From Transfer Functions to State Space

In the 1950s, control engineers exclusively used **transfer functions**, which relate outputs to inputs in the frequency domain via Laplace transforms. Transfer functions are elegant for single-input single-output (SISO) systems and for analyzing stability via Nyquist diagrams and Bode plots. However, they fail catastrophically for multi-input multi-output (MIMO) systems and provide no insight into what is happening *inside* the system.

In 1960, **Rudolf E. Kalman** published his seminal paper [**?**] introducing state-space representations and the foundational concepts of **controllability** and **observability**. This was revolutionary: it enabled the design of optimal controllers for complex systems, the treatment of MIMO systems, and the development of the Kalman filter—one of the most important algorithms in engineering. The state-space framework was so powerful that it became the universal language of control theory.

The fundamental advantage of state-space methods over transfer functions is *transparency* [**?**]. A transfer function $G(s) = \frac{Y(s)}{U(s)}$ tells you how the input drives the output, but it hides the internal structure. A state-space model $\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu}$ exposes the entire flow of energy and information through the system. This transparency makes it possible to:

- Design controllers that regulate internal state, not just final outputs
- Detect when a system is uncontrollable or unobservable
- Handle systems with multiple inputs and multiple outputs naturally
- Compute optimal control laws analytically
- Estimate hidden state variables from measurements (Kalman filter)

The rest of this chapter builds the state-space framework from first principles. By the end, you will understand why Kalman's insight was revolutionary and why state space is now the only way modern control engineers think.

2.3 Constructing a State Vector

Let us build a state space from the ground up using the simplest dynamical system possible: a 1-dimensional point mass (like a train on a straight track).

We want to predict what the train will do. To do this, we need to know its position, q. Is q the "state" of the system?

No. If we only know that the train is at $q = 100$ meters, we cannot predict where it will be 1 second from now. It could be moving right, moving left, or stationary.

Therefore, to complete our understanding, we must also know its velocity, $\dot{q}$ (the time-derivative of position). If we know $(q, \dot{q})$, we know its exact location and its exact speed.

Do we need to know acceleration ($\ddot{q}$) as part of the state? According to Newton's Second Law: $F = m\ddot{q}$. Acceleration is not an independent property of the train; it is completely driven by the external forces applied at that instant. Therefore, if we know the forces (the Control Inputs, $\mathbf{u}$), we can immediately compute the acceleration. The state itself is fully defined by purely the position and velocity.

Definition 2.1. The State Vector x

The state vector $\mathbf{x}$ is the minimum set of mathematical variables completely describing the condition of a dynamic system at a specific point in time.
For our train, it is a 2-dimensional column vector:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} q \\ \dot{q} \end{bmatrix} = \begin{bmatrix} \text{position} \\ \text{velocity} \end{bmatrix} \tag{2.1}$$

For a general system with n degrees of freedom, the state vector lives in n-dimensional state space: $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^n$.

If you take a photograph of a system, you capture its position $\mathbf{q}$. But a state vector $\mathbf{x}$ is the "ultimate photograph"—it captures the position *and* the velocity. With $\mathbf{x}$ and the laws of physics, the entire future of the system is deterministic.

2.3.1 Worked Example 1: Simple Pendulum

Example 2.1. State Vector for a Simple Pendulum

A pendulum of length L and mass m is suspended from a pivot. Let θ denote the angle from vertical, and τ denote an applied torque at the pivot.
The equation of motion is:

$$mL^2\ddot{\theta} + mgL\sin(\theta) = \tau \tag{2.2}$$

To predict the pendulum's future, what do we need?

- The angle θ (where is it pointing?)
- The angular velocity $\dot{\theta}$ (is it spinning?)
- The torque τ is the control input (provided at the moment we want to predict)

Thus the state vector is:

$$\mathbf{x} = \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix} \tag{2.3}$$

This is a 2-dimensional state space, even though the pendulum physically swings in a 1-dimensional arc in our visual space.

2.3.2 Worked Example 2: DC Motor

Example 2.2. State Vector for a DC Motor

A DC motor is driven by input voltage V_{in}. Let θ be the rotor angle, and let i be the current flowing through the motor windings.
The governing equations are:

$$L\frac{\mathrm{d}i}{\mathrm{d}t} = V_{\text{in}} - Ri - K_e\dot{\theta} \tag{2.4}$$

$$J\frac{\mathrm{d}\dot{\theta}}{\mathrm{d}t} = K_t i - b\dot{\theta} \tag{2.5}$$

where:

- L is the motor inductance, R is resistance
- K_e is the back-EMF constant, K_t is the torque constant
- J is the rotor inertia, b is viscous friction

The state vector must include all variables whose time-derivatives depend on the inputs. These are:

$$\mathbf{x} = \begin{bmatrix} i \\ \theta \\ \dot{\theta} \end{bmatrix} \tag{2.6}$$

This is a 3-dimensional state space. Notice we include θ (rotor position) even though it does not directly appear in the governing equations—we need it to fully specify the motor's configuration, and its time-derivative $\dot{\theta}$ does appear.

2.3.3 Worked Example 3: Two-Tank Fluid System

Example 2.3. State Vector for Two Coupled Tanks

Two tanks, each with cross-sectional area A, are connected by a valve. Let h_1 and h_2 be the fluid levels. Flow into tank 1 is Q_{in}, and flow between tanks is proportional to the height difference:

$$Q_{1\to 2} = C\sqrt{|h_1 - h_2|} \tag{2.7}$$

where C is a flow coefficient, and outflow from tank 2 is:

$$Q_{\text{out}} = C_{\text{out}}\sqrt{h_2} \tag{2.8}$$

The height rates of change are:

$$A\frac{\mathrm{d}h_1}{\mathrm{d}t} = Q_{\text{in}} - Q_{1\to 2} \tag{2.9}$$

$$A\frac{\mathrm{d}h_2}{\mathrm{d}t} = Q_{1\to 2} - Q_{\text{out}} \tag{2.10}$$

To predict the system, we need the current heights of both tanks. The state vector is:

$$\mathbf{x} = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} \tag{2.11}$$

This is a 2-dimensional state space. The control input is the inflow rate Q_{in}.

2.4 Converting n-th Order ODEs to State Form

Theorem 2.1. Conversion of n-th Order ODE to First-Order State Form

Any n-th order differential equation of the form

$$y^{(n)} + a_{n-1}y^{(n-1)} + \cdots + a_1\dot{y} + a_0 y = u(t) \tag{2.12}$$

can be converted to a system of n first-order ODEs via state-vector substitution.

Proof. Define the state variables as consecutive derivatives of y:

$$x_1 = y \tag{2.13}$$
$$x_2 = \dot{y} \tag{2.14}$$
$$x_3 = \ddot{y} \tag{2.15}$$
$$\vdots \tag{2.16}$$
$$x_n = y^{(n-1)} \tag{2.17}$$

Then:

$$\dot{x}_1 = x_2 \tag{2.18}$$
$$\dot{x}_2 = x_3 \tag{2.19}$$
$$\vdots \tag{2.20}$$
$$\dot{x}_{n-1} = x_n \tag{2.21}$$
$$\dot{x}_n = u(t) - a_0 x_1 - a_1 x_2 - \cdots - a_{n-1}x_n \tag{2.22}$$

where the final equation is obtained by solving the original ODE for $y^{(n)} = \dot{x}_n$:

$$\dot{x}_n = u(t) - a_0 x_1 - a_1 x_2 - \cdots - a_{n-1}x_n \tag{2.23}$$

In matrix form, this is $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$ where $\mathbf{A}$ is the companion matrix:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ -a_0 & -a_1 & -a_2 & \cdots & -a_{n-1} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \tag{2.24}$$

This representation is unique up to change of state variables (i.e., state transformation). □

2.5 State Space Representations

Because the state vector $\mathbf{x}$ contains n variables, it naturally lives in an n-dimensional geometry: the State Space, denoted as $\mathcal{X} \subset \mathbb{R}^n$.

An astonishing realization for beginners is that when a dynamic system (like an airplane or a robotic arm) moves through physical space over time, its entire sprawling motion is represented as a single, unified **1-dimensional curve** snaking through n-dimensional state space. This curve is called the system's **Trajectory** $\mathcal{T}$.

2.5.1 Standard Form Differential Equations

The immense power of state space is that it allows us to convert complex, higher-order physical differential equations into simple, first-order vector math.

Consider a simple mass-spring-damper system governed by a second-order differential equation:

$$m\ddot{q} + c\dot{q} + kq = u \tag{2.25}$$

where q is position, m is mass, c is damping, k is stiffness, and u is our control force (e.g., a motor pushing the mass).

By defining our state vector as $\mathbf{x} = [x_1, x_2]^T = [q, \dot{q}]^T$, we can rewrite this physical equation as purely mathematical first-order derivatives. First, isolate the highest derivative ($\ddot{q}$):

$$\ddot{q} = \frac{1}{m}(u - c\dot{q} - kq) \tag{2.26}$$

Now, substitute the state variables ($x_1 = q, x_2 = \dot{q}$):

$$\dot{x}_1 = x_2 \tag{2.27}$$

$$\dot{x}_2 = \frac{1}{m}(u - cx_2 - kx_1) \tag{2.28}$$

This is called the **State Space Representation**. It is universally written in nonlinear control literature as:

Mathematical Principle 2.1. The Fundamental Equation of Control Theory

Every autonomous control system can be written as a first-order vector differential equation mapping the current state and current control commands to the velocity of the state vector:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, t) \tag{2.29}$$

This vector field f dictates how the system "flows" through the abstract geometry of the state space.

2.5.2 Linear State Space: The A and B Matrices

When the system is perfectly linear (like the mass-spring-damper above), the function $f(\mathbf{x}, \mathbf{u}, t)$ can be split into two matrices: the Dynamics Matrix ($\mathbf{A}$) and the Input Matrix ($\mathbf{B}$).

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \tag{2.30}$$

If we package our mass-spring equations into matrices, we get:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u \tag{2.31}$$

- **The A Matrix**: $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}$. This matrix defines the "drift" of the system. If you turn off all motors ($u = 0$), the $\mathbf{A}$ matrix dictates exactly how the system will coast to a stop, oscillate, or explode. Its eigenvalues completely define the system's natural stability!

- **The B Matrix**: $\mathbf{B} = \begin{bmatrix} 0 \\ 1/m \end{bmatrix}$. This matrix defines how your control inputs u actually inject energy into the state. Notice that the top value is 0, meaning our motor cannot instantly teleport the position (x_1); it can only directly push the velocity (x_2).

2.6 Output Equations and Observability

Often, we cannot measure the entire state vector. For example, in a DC motor, we can measure the voltage and current flowing through it, but we may not directly measure the angular position θ. The **output equation** describes what we actually measure:

Definition 2.2. Output Equation

For a linear system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, the output equation is:

$$\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u} \tag{2.32}$$

where:

- $\mathbf{y} \in \mathbb{R}^p$ is the measurement vector (what we can observe)
- $\mathbf{C} \in \mathbb{R}^{p \times n}$ is the output matrix
- $\mathbf{D} \in \mathbb{R}^{p \times m}$ is the feedthrough matrix (often zero)

The question "Can we infer the full state $\mathbf{x}$ from the measurements $\mathbf{y}$?" is precisely the question of **observability**, which we formalize later in Section 2.11.

2.7 Equilibrium Points and Linearization

2.7.1 Equilibrium Point Definition

Definition 2.3. Equilibrium Point

An **equilibrium point** (or fixed point) $\mathbf{x}^* \in \mathbb{R}^n$ of the system $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}^*)$ is a point where the vector field is zero:

$$f(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{0} \tag{2.33}$$

At an equilibrium, the state does not change; the system is in a "steady state."

For linear systems $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, if $\mathbf{A}$ is invertible and $\mathbf{u}^* = \mathbf{0}$, the unique equilibrium is $\mathbf{x}^* = \mathbf{0}$. For nonlinear systems, there may be multiple equilibria.

2.7.2 Linearization via Taylor Expansion

To understand the local behavior near an equilibrium, we linearize via Taylor expansion. Suppose we have an equilibrium $\mathbf{x}^*$ at control input $\mathbf{u}^*$. Consider a small perturbation $\boldsymbol{\delta x} = \mathbf{x} - \mathbf{x}^*$ and $\boldsymbol{\delta u} = \mathbf{u} - \mathbf{u}^*$.

Expand f in a Taylor series around $(\mathbf{x}^*, \mathbf{u}^*)$:

$$f(\mathbf{x}^* + \boldsymbol{\delta x}, \mathbf{u}^* + \boldsymbol{\delta u}) = f(\mathbf{x}^*, \mathbf{u}^*) + \mathbf{J}_f \boldsymbol{\delta x} + \mathbf{J}_u \boldsymbol{\delta u} + O(\|\boldsymbol{\delta x}\|^2, \|\boldsymbol{\delta u}\|^2) \tag{2.34}$$

where $\mathbf{J}_f$ is the **Jacobian matrix** of f with respect to the state:

$$\mathbf{J}_f = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{(\mathbf{x}^*, \mathbf{u}^*)} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}_{(\mathbf{x}^*, \mathbf{u}^*)} \tag{2.35}$$

and similarly:

$$\mathbf{J}_u = \left.\frac{\partial f}{\partial \mathbf{u}}\right|_{(\mathbf{x}^*, \mathbf{u}^*)} \tag{2.36}$$

Since $f(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{0}$, the linearized dynamics are:

$$\dot{\boldsymbol{\delta x}} \approx \mathbf{A}_c \boldsymbol{\delta x} + \mathbf{B}_c \boldsymbol{\delta u} \tag{2.37}$$

where $\mathbf{A}_c = \mathbf{J}_f$ and $\mathbf{B}_c = \mathbf{J}_u$. This is a linear system describing the small-signal behavior around the equilibrium.

Example 2.4. Pendulum Linearization

Consider the pendulum from Example 2.3.1 with $\tau = 0$ (no control). The equation is:

$$\ddot{\theta} + \frac{g}{L}\sin(\theta) = 0 \tag{2.38}$$

State vector: $\mathbf{x} = [\theta, \dot{\theta}]^T$. The vector field is:

$$f(\mathbf{x}) = \begin{bmatrix} x_2 \\ -\frac{g}{L}\sin(x_1) \end{bmatrix} \tag{2.39}$$

Equilibria: $f(\mathbf{x}^*) = \mathbf{0}$ when $x_2 = 0$ and $\sin(x_1) = 0$, so $\mathbf{x}^* = [k\pi, 0]^T$ for $k \in \mathbb{Z}$.
Bottom equilibrium ($\theta = 0$): The Jacobian is:

$$\mathbf{J}_f = \begin{bmatrix} 0 & 1 \\ -\frac{g}{L}\cos(\theta) & 0 \end{bmatrix}\Bigg|_{\theta=0} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{L} & 0 \end{bmatrix} \tag{2.40}$$

The linearized system near the bottom is $\ddot{\theta} = -\frac{g}{L}\theta$, a harmonic oscillator.
Top equilibrium ($\theta = \pi$):

$$\mathbf{J}_f\Big|_{\theta=\pi} = \begin{bmatrix} 0 & 1 \\ \frac{g}{L} & 0 \end{bmatrix} \tag{2.41}$$

The linearized system is $\ddot{\theta} = \frac{g}{L}\theta$, which has exponentially growing solutions—an unstable saddle.

2.8 Phase Portraits and Stability Classification

For a 2D system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$, the phase portrait is the collection of all trajectories in the (x_1, x_2) plane. The behavior depends entirely on the eigenvalues of $\mathbf{A}$.

Theorem 2.2. Stability Classification from Eigenvalues

For a 2D linear system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ with eigenvalues λ_1, λ_2:

- **Stable Node**: Both eigenvalues are real and negative ($\lambda_1, \lambda_2 < 0$). Trajectories spiral into the origin monotonically.
- **Unstable Node**: Both eigenvalues are real and positive ($\lambda_1, \lambda_2 > 0$). Trajectories spiral away from the origin.
- **Saddle Point**: One eigenvalue is positive, one is negative. Trajectories approach along the stable manifold (eigenvector of $\lambda < 0$) and diverge along the unstable manifold (eigenvector of $\lambda > 0$).
- **Stable Spiral (Focus)**: Eigenvalues are complex conjugates with negative real part ($\lambda = \alpha \pm i\beta$, $\alpha < 0$). Trajectories spiral inward.
- **Unstable Spiral**: Eigenvalues are complex conjugates with positive real part. Trajectories spiral outward.
- **Center**: Eigenvalues are purely imaginary ($\lambda = \pm i\omega$). Trajectories form closed orbits (periodic motion).

The origin is **asymptotically stable** if and only if all eigenvalues have negative real part. The origin is **marginally stable** if the largest real part is zero. The origin is **unstable** if any eigenvalue has positive real part.

2.9 Solution of Linear Systems: The Matrix Exponential

For a linear time-invariant (LTI) system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ with initial condition $\mathbf{x}(0) = \mathbf{x}_0$, the solution is:

Theorem 2.3. Solution via Matrix Exponential

The unique solution is:

$$\mathbf{x}(t) = e^{\mathbf{A}t}\mathbf{x}_0 \tag{2.42}$$

where $e^{\mathbf{A}t}$ is the **matrix exponential**, defined as:

$$e^{\mathbf{A}t} = \sum_{k=0}^{\infty} \frac{(\mathbf{A}t)^k}{k!} = \mathbf{I} + \mathbf{A}t + \frac{(\mathbf{A}t)^2}{2!} + \frac{(\mathbf{A}t)^3}{3!} + \cdots \tag{2.43}$$

Proof. Consider the scalar ODE $\dot{x} = ax$ with initial condition $x(0) = x_0$. The solution is $x(t) = e^{at}x_0$.

By analogy, define $e^{\mathbf{A}t}$ as the solution to:

$$\frac{\mathrm{d}}{\mathrm{d}t}e^{\mathbf{A}t} = \mathbf{A}e^{\mathbf{A}t}, \quad e^{\mathbf{A}\cdot 0} = \mathbf{I} \tag{2.44}$$

The Taylor series ansatz:

$$e^{\mathbf{A}t} = \sum_{k=0}^{\infty} \frac{(\mathbf{A}t)^k}{k!} \tag{2.45}$$

satisfies this differential equation because:

$$\frac{\mathrm{d}}{\mathrm{d}t}\sum_{k=0}^{\infty} \frac{(\mathbf{A}t)^k}{k!} = \sum_{k=1}^{\infty} \frac{k(\mathbf{A}t)^{k-1}\mathbf{A}}{k!} = \sum_{k=1}^{\infty} \frac{(\mathbf{A}t)^{k-1}\mathbf{A}}{(k-1)!} = \mathbf{A}\sum_{j=0}^{\infty} \frac{(\mathbf{A}t)^j}{j!} = \mathbf{A}e^{\mathbf{A}t} \tag{2.46}$$

with $e^{\mathbf{A}\cdot 0} = \mathbf{I}$. Now verify that $\mathbf{x}(t) = e^{\mathbf{A}t}\mathbf{x}_0$ solves the original system:

$$\dot{\mathbf{x}}(t) = \frac{\mathrm{d}}{\mathrm{d}t}\left(e^{\mathbf{A}t}\mathbf{x}_0\right) = \mathbf{A}e^{\mathbf{A}t}\mathbf{x}_0 = \mathbf{A}\mathbf{x}(t) \quad \checkmark \tag{2.47}$$

□

Practical Computation: For small-dimensional systems, the matrix exponential can be computed via diagonalization. If $\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^{-1}$ where $\mathbf{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$, then:

$$e^{\mathbf{A}t} = \mathbf{P}e^{\mathbf{\Lambda}t}\mathbf{P}^{-1} = \mathbf{P}\mathrm{diag}(e^{\lambda_1 t}, \ldots, e^{\lambda_n t})\mathbf{P}^{-1} \tag{2.48}$$

For systems with input, $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}(t)$:

$$\mathbf{x}(t) = e^{\mathbf{A}t}\mathbf{x}_0 + \int_0^t e^{\mathbf{A}(t-\tau)}\mathbf{B}\mathbf{u}(\tau)\mathrm{d}\tau \tag{2.49}$$

This is the superposition of free response (first term) and forced response (second term).

2.10 Controllability

The key question in control is: *Can we steer the system from any initial state to any desired final state using our inputs?*

Definition 2.4. Controllability

A linear system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ is **controllable** if for any initial state $\mathbf{x}_0$ and any final state $\mathbf{x}_f$, there exists a finite time $T > 0$ and an input sequence $\mathbf{u}(t)$ for $t \in [0, T]$ that drives the system from $\mathbf{x}_0$ to $\mathbf{x}_f$.

Theorem 2.4. Kalman Controllability Rank Condition

The system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ is controllable if and only if the **controllability matrix**

$$\mathbf{C}_{\text{rank}} = \begin{bmatrix} \mathbf{B} & \mathbf{A}\mathbf{B} & \mathbf{A}^2\mathbf{B} & \cdots & \mathbf{A}^{n-1}\mathbf{B} \end{bmatrix} \tag{2.50}$$

has full rank: $\text{rank}(\mathbf{C}_{\text{rank}}) = n$.

Sketch. The controllability matrix $\mathbf{C}_{\text{rank}}$ has dimensions $n \times nm$. Each column $\mathbf{A}^k\mathbf{B}$ represents the "reach" of the input after k applications of the dynamics. The rank condition ensures that the span of these columns covers all n dimensions of state space.

The intuition is from the Cayley-Hamilton theorem: any power $\mathbf{A}^k$ for $k \geq n$ can be expressed as a linear combination of $\mathbf{A}^0, \mathbf{A}^1, \ldots, \mathbf{A}^{n-1}$. Thus, checking columns up to $\mathbf{A}^{n-1}\mathbf{B}$ is sufficient.

A rigorous proof uses the Gram matrix approach: controllability is equivalent to the controllability Gram matrix

$$\mathbf{W}_c = \int_0^T e^{\mathbf{A}\tau}\mathbf{B}\mathbf{B}^T e^{\mathbf{A}^T\tau} \mathrm{d}\tau \tag{2.51}$$

being positive definite, which is equivalent to $\mathbf{C}_{\text{rank}}$ having full rank. □

Physical Interpretation: If the system is not controllable, it means some mode or degree of freedom cannot be directly influenced by the available inputs. For a DC motor whose position is θ and current is i, if the only measurement is current and there is no direct control over electrical resistance, the position might be uncontrollable.

2.11 Observability

Observability is the dual of controllability: *Can we infer the full state from measurements alone?*

Definition 2.5. Observability

A linear system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, $\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u}$ is **observable** if the initial state $\mathbf{x}(0)$ can be uniquely determined from knowledge of the input $\mathbf{u}(t)$ and the output $\mathbf{y}(t)$ over a finite time interval $[0, T]$.

Theorem 2.5. Kalman Observability Rank Condition

The system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}+\mathbf{B}\mathbf{u}$, $\mathbf{y} = \mathbf{C}\mathbf{x}+\mathbf{D}\mathbf{u}$ is observable if and only if the **observability matrix**

$$\mathbf{O} = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \mathbf{CA}^2 \\ \vdots \\ \mathbf{CA}^{n-1} \end{bmatrix} \tag{2.52}$$

has full rank: $\text{rank}(\mathbf{O}) = n$.

The observability matrix has dimensions $np \times n$ where p is the number of measurements. Each row $\mathbf{CA}^k$ represents the k-th time derivative of the measurement, weighted by the dynamics. If these rows span all n dimensions, we can reconstruct the state.

2.12 The Mass-Spring-Damper System: Complete Analysis

Let us apply the concepts from this chapter to the mass-spring-damper system with stiffness k, damping c, and mass m:

$$m\ddot{q} + c\dot{q} + kq = u \tag{2.53}$$

State vector: $\mathbf{x} = [q, \dot{q}]^T$.

Matrices:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 1/m \end{bmatrix} \tag{2.54}$$

Characteristic polynomial:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = \det \begin{bmatrix} -\lambda & 1 \\ -k/m & -c/m - \lambda \end{bmatrix} = \lambda^2 + \frac{c}{m}\lambda + \frac{k}{m} \tag{2.55}$$

Eigenvalues:

$$\lambda_{1,2} = -\frac{c}{2m} \pm \sqrt{\left(\frac{c}{2m}\right)^2 - \frac{k}{m}} = -\frac{c}{2m} \pm \sqrt{\frac{c^2}{4m^2} - \frac{k}{m}} \tag{2.56}$$

Define the natural frequency $\omega_0 = \sqrt{k/m}$ and damping ratio $\zeta = \frac{c}{2\sqrt{km}}$. Then:

$$\lambda_{1,2} = -\zeta\omega_0 \pm i\omega_0\sqrt{1-\zeta^2} \tag{2.57}$$

(assuming $\zeta < 1$ for the underdamped case; see below for other cases).

2.12.1 Underdamped ($0 < \zeta < 1$)

The discriminant is negative, so eigenvalues are complex: $\lambda = -\zeta\omega_0 \pm i\omega_d$ where $\omega_d = \omega_0\sqrt{1-\zeta^2}$ is the damped frequency.

The mass oscillates while decaying. The solution is:

$$q(t) = e^{-\zeta\omega_0 t}\left(q_0\cos(\omega_d t) + \frac{\dot{q}_0 + \zeta\omega_0 q_0}{\omega_d}\sin(\omega_d t)\right) \tag{2.58}$$

The trajectory in phase space is a **stable spiral**, spiraling into the origin.

2.12.2 Critically Damped ($\zeta = 1$)

The eigenvalues coincide: $\lambda_1 = \lambda_2 = -\omega_0$. The matrix $\mathbf{A}$ is not diagonalizable; it has a Jordan block.

The solution is:

$$q(t) = (q_0 + (\dot{q}_0 + \omega_0 q_0)t)e^{-\omega_0 t} \tag{2.59}$$

This is the fastest non-oscillatory return to equilibrium. The trajectory does *not* spiral; it approaches the origin monotonically.

2.12.3 Overdamped ($\zeta > 1$)

The eigenvalues are real and distinct:

$$\lambda_1 = -\zeta\omega_0 + \omega_0\sqrt{\zeta^2 - 1}, \quad \lambda_2 = -\zeta\omega_0 - \omega_0\sqrt{\zeta^2 - 1} \tag{2.60}$$

Both are negative (since $\zeta > 0$). The solution is:

$$q(t) = C_1 e^{\lambda_1 t} + C_2 e^{\lambda_2 t} \tag{2.61}$$

The slower eigenvalue λ_1 dominates for large times. The trajectory is a **stable node**—no oscillation, just exponential decay. The system returns to rest more slowly than the critically damped case.

2.12.4 Controllability and Observability

The controllability matrix is:

$$\mathbf{C}_{\text{rank}} = \begin{bmatrix} \mathbf{B} & \mathbf{AB} \end{bmatrix} = \begin{bmatrix} 0 & 1/m \\ 1/m & -c/m^2 \end{bmatrix} \tag{2.62}$$

Its determinant is $\det(\mathbf{C}_{\text{rank}}) = 0 - \frac{1}{m^2} = -\frac{1}{m^2} \neq 0$. Thus $\mathbf{C}_{\text{rank}}$ has full rank and the system is **controllable**. We can steer position and velocity to any desired values using the input force u.

If we measure position only, $\mathbf{C} = [1, 0]$, the observability matrix is:

$$\mathbf{O} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{2.63}$$

which has rank 2. The system is **observable**. We can infer both position and velocity from position measurements alone (by differentiating or using a Kalman filter).

2.13 Transfer Function to State Space and Back

The transfer function $G(s) = \frac{Y(s)}{U(s)}$ relates input and output in the Laplace domain. For a linear system with zero initial conditions, the transfer function is:

$$G(s) = \mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B} + \mathbf{D} \tag{2.64}$$

This is derived from the state-space equations by taking Laplace transforms:

$$s\mathbf{X}(s) = \mathbf{A}\mathbf{X}(s) + \mathbf{B}\mathbf{U}(s) \tag{2.65}$$
$$\mathbf{Y}(s) = \mathbf{C}\mathbf{X}(s) + \mathbf{D}\mathbf{U}(s) \tag{2.66}$$

Solve for $\mathbf{X}(s)$:

$$(s\mathbf{I} - \mathbf{A})\mathbf{X}(s) = \mathbf{B}\mathbf{U}(s) \quad \Rightarrow \quad \mathbf{X}(s) = (s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}\mathbf{U}(s) \tag{2.67}$$

Substitute into the output equation:

$$\mathbf{Y}(s) = \left[\mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B} + \mathbf{D}\right]\mathbf{U}(s) = G(s)\mathbf{U}(s) \tag{2.68}$$

Conversion from transfer function to state space is not unique—there are many valid state representations. The most common choice is the controllable canonical form (the companion matrix form from Section 2.4).

2.13.1 Discrete-Time State Space

Every modern robot controller is implemented on a digital processor that reads sensors and writes actuator commands at a fixed sample rate $f_s = 1/T_s$. Between samples the controller holds its output constant (zero-order hold). We therefore need a *discrete-time* counterpart of the continuous state-space model.

Definition 2.6. Discrete-Time State-Space Model

A linear discrete-time system is

$$\mathbf{x}[k+1] = \mathbf{A}_d\,\mathbf{x}[k] + \mathbf{B}_d\,\mathbf{u}[k], \tag{2.69}$$
$$\mathbf{y}[k] = \mathbf{C}\,\mathbf{x}[k] + \mathbf{D}\,\mathbf{u}[k], \tag{2.70}$$

where $k \in \{0, 1, 2, \ldots\}$ indexes the sample instants $t_k = k\,T_s$.

Exact discretization (zero-order hold). Given the continuous system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ and a sampling period T_s, the exact discrete-time matrices are

$$\mathbf{A}_d = e^{\mathbf{A}\,T_s}, \qquad \mathbf{B}_d = \int_0^{T_s} e^{\mathbf{A}\,\tau}\,\mathbf{B}\,\mathrm{d}\tau = \mathbf{A}^{-1}\big(e^{\mathbf{A}\,T_s} - \mathbf{I}\big)\mathbf{B}, \tag{2.71}$$

where the closed-form expression for $\mathbf{B}_d$ holds when $\mathbf{A}$ is invertible. The matrix exponential $e^{\mathbf{A}\,T_s}$ connects directly to Theorem 2.3: the free response over one sample period is simply propagated by $\mathbf{A}_d$.

Theorem 2.6. Discrete-Time Stability Criterion

The equilibrium $\mathbf{x}^* = \mathbf{0}$ of the autonomous discrete-time system $\mathbf{x}[k+1] = \mathbf{A}_d\,\mathbf{x}[k]$ is asymptotically stable if and only if every eigenvalue of $\mathbf{A}_d$ lies strictly inside the unit

circle:

$$|\lambda_i(\mathbf{A}_d)| < 1 \quad \forall\, i. \tag{2.72}$$

This is the discrete analogue of requiring all eigenvalues of $\mathbf{A}$ to have negative real parts in continuous time (Theorem 2.2).

In Plain Language 2.3. All Robot Controllers Are Discrete

All modern robot controllers run in discrete time at sample rates of 1–10 kHz. The continuous-time model $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ is the physicist's description of reality; the discrete-time model $\mathbf{x}[k{+}1] = \mathbf{A}_d\mathbf{x}[k] + \mathbf{B}_d\mathbf{u}[k]$ is what actually executes on the real-time processor. A control design that ignores this distinction—for example, by using too low a sample rate so that eigenvalues migrate outside the unit circle—will destabilise an otherwise stable system.

2.14 Nonlinear Systems and Phase Portraits

While we have focused on linear systems, real physical systems are often nonlinear. For a nonlinear system $\dot{\mathbf{x}} = f(\mathbf{x})$, the phase portrait can be qualitatively understood by:

1. Finding equilibrium points by solving $f(\mathbf{x}^*) = \mathbf{0}$
2. Linearizing around each equilibrium: $\dot{\boldsymbol{\delta x}} \approx \mathbf{J}_f(\mathbf{x}^*)\boldsymbol{\delta x}$
3. Classifying the equilibrium by its eigenvalues (Theorem 2.8)
4. Plotting the vector field $\dot{\mathbf{x}} = f(\mathbf{x})$ to visualize global behavior

The linearization is valid in a neighborhood of the equilibrium. For large perturbations, we must rely on numerical simulation or other techniques (Lyapunov theory, bifurcation analysis).

2.15 Python Worked Example: State Space Simulation and Phase Portrait

Consider the underdamped mass-spring-damper system: $m = 1$, $c = 0.5$, $k = 2$, with initial conditions $q_0 = 1$, $\dot{q}_0 = 0$.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# Parameters
m, c, k = 1.0, 0.5, 2.0
A = np.array([[0, 1], [-k/m, -c/m]])
B = np.array([[0], [1/m]])
```

```
# System dynamics
def dynamics(x, t, u=0):
    return A @ x + B.flatten() * u

# Initial condition
x0 = np.array([1.0, 0.0])
t = np.linspace(0, 10, 1000)

# Simulate
x = odeint(dynamics, x0, t)

# Phase portrait
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Time evolution
ax1.plot(t, x[:, 0], label='Position q(t)')
ax1.plot(t, x[:, 1], label='Velocity dq/dt(t)')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('State')
ax1.legend()
ax1.grid()
ax1.set_title('Time Evolution')

# Phase portrait
ax2.plot(x[:, 0], x[:, 1], 'b-', linewidth=2)
ax2.plot(x[0, 0], x[0, 1], 'go', markersize=10, label='Initial')
ax2.plot(x[-1, 0], x[-1, 1], 'ro', markersize=10, label='Final')
ax2.set_xlabel('Position q')
ax2.set_ylabel('Velocity dq/dt')
ax2.grid()
ax2.legend()
ax2.set_title('Phase Portrait (Stable Spiral)')

plt.tight_layout()
plt.show()

# Eigenvalues
eigenvalues, eigenvectors = np.linalg.eig(A)
print(f"Eigenvalues: {eigenvalues}")
print(f"Damping ratio zeta = {c / (2*np.sqrt(k*m)):.3f}")
print(f"Natural freq omega_0 = {np.sqrt(k/m):.3f} rad/s")
```

This code simulates the system and plots both the time evolution and the phase portrait. For the underdamped case ($\zeta = 0.177$), you should see a spiral converging to the origin with oscillation.

2.16 Equilibrium Analysis Summary

For any equilibrium point $\mathbf{x}^*$:

- Compute the Jacobian: $\mathbf{A} = \frac{\partial f}{\partial \mathbf{x}}\big|_{\mathbf{x}^*}$
- Find eigenvalues of $\mathbf{A}$
- Classify the equilibrium according to Theorem 2.8
- If all eigenvalues have negative real part: **asymptotically stable**
- If any eigenvalue has positive real part: **unstable**
- If the largest real part is zero (center or boundary case): **marginally stable**

2.17 Summary

Mathematical Principle 2.2. Key Takeaways: State Space Fundamentals

1. State Vector: The state $\mathbf{x}$ is the minimum information needed to predict the system's future. For an n-th order ODE, the state is n-dimensional.
2. Kalman's Revolution (1960): State-space representations replace transfer functions, enabling control of MIMO systems, design of optimal controllers, and systematic analysis of internal system structure.
3. Standard Forms: Any system can be written as $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, t)$ (nonlinear) or $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ (linear). The $\mathbf{A}$ matrix encodes natural stability; the $\mathbf{B}$ matrix encodes control authority.
4. ODE Reduction: Any n-th order ODE reduces to n first-order state equations via companion form.
5. Equilibrium and Linearization: Equilibria are found by solving $f(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{0}$. Local behavior is determined by the Jacobian $\mathbf{J}_f = \partial f / \partial \mathbf{x}$.
6. Stability from Eigenvalues: Eigenvalues of $\mathbf{A}$ determine phase-portrait structure (node, spiral, saddle, center). Negative real parts mean stability.
7. Matrix Exponential: The solution $\mathbf{x}(t) = e^{\mathbf{A}t}\mathbf{x}_0 + \int_0^t e^{\mathbf{A}(t-\tau)}\mathbf{B}\mathbf{u}(\tau)\,\mathrm{d}\tau$ decomposes into free response and forced response.
8. Controllability (Kalman Rank Test): The system is controllable iff $\operatorname{rank}([\mathbf{B}, \mathbf{AB}, \ldots, \mathbf{A}^{n-1}\mathbf{B}]) = n$. If not controllable, some state variables cannot be influenced by inputs.
9. Observability (Dual of Controllability): The system is observable iff $\operatorname{rank}([\mathbf{C}^T, \mathbf{A}^T\mathbf{C}^T, \ldots, (\mathbf{A}^T)^{n-1}\mathbf{C}^T]^T) = n$. If not observable, some state variables cannot be inferred from measurements.
10. Duality: A system is controllable iff its dual (with $\mathbf{A}^T$, $\mathbf{C}^T$, $\mathbf{B}^T$) is observable. This duality underpins Kalman filter design.

2.18 Lyapunov Stability Theory

The eigenvalue-based stability analysis of Theorem 2.2 is powerful but inherently *local*: it applies only in a neighbourhood of an equilibrium where the linearization is valid. For nonlinear systems, we need a theory that can certify stability over large—possibly global—regions of state space. **Lyapunov's direct method** provides exactly this by replacing spectral analysis with an energy-like argument.

Linearization tells us about local stability. Lyapunov theory works globally.

Definition 2.7. Lyapunov Function $V(\mathbf{x})$

Consider the autonomous system $\dot{\mathbf{x}} = f(\mathbf{x})$ with an equilibrium at $\mathbf{x}^* = \mathbf{0}$. A continuously differentiable function $V : \mathbb{R}^n \to \mathbb{R}$ is called a **Lyapunov function candidate** if

1. $V(\mathbf{0}) = 0$ (zero at the equilibrium),
2. $V(\mathbf{x}) > 0$ for all $\mathbf{x} \neq \mathbf{0}$ in a neighbourhood of the origin (*positive definite*).

Its time derivative along trajectories of the system is

$$\dot{V}(\mathbf{x}) = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}) = \nabla V(\mathbf{x})^T f(\mathbf{x}). \tag{2.73}$$

Theorem 2.7. Lyapunov's Direct Method (Second Method)

Let $\mathbf{x}^* = \mathbf{0}$ be an equilibrium of $\dot{\mathbf{x}} = f(\mathbf{x})$ and let $V(\mathbf{x})$ be a Lyapunov function candidate in a domain $\mathcal{D}$ containing the origin.

1. If $\dot{V}(\mathbf{x}) \leq 0$ in $\mathcal{D}$, the equilibrium is **stable** (in the sense of Lyapunov).
2. If $\dot{V}(\mathbf{x}) < 0$ for all $\mathbf{x} \neq \mathbf{0}$ in $\mathcal{D}$, the equilibrium is **asymptotically stable**.
3. If the conditions above hold with $\mathcal{D} = \mathbb{R}^n$ and, in addition, $V(\mathbf{x}) \to \infty$ as $\|\mathbf{x}\| \to \infty$ (*radially unbounded*), the equilibrium is **globally asymptotically stable**.

The power of the theorem lies in the fact that we never need to solve the differential equation—we only need to find a single scalar function V with the right properties.

Example 2.5. Pendulum with Damping

Consider a simple pendulum of mass m and length ℓ with viscous damping $b > 0$:

$$m\ell^2\ddot{\theta} + b\dot{\theta} + mg\ell\sin\theta = 0. \tag{2.74}$$

The state is $\mathbf{x} = (\theta,\ \dot{\theta})^T$ and the downward equilibrium is $\mathbf{x}^* = \mathbf{0}$.
Choose the total mechanical energy as a Lyapunov candidate:

$$V(\theta, \dot{\theta}) = \underbrace{\tfrac{1}{2}\, m\ell^2\, \dot{\theta}^2}_{\text{kinetic}} + \underbrace{mg\ell\,(1 - \cos\theta)}_{\text{potential}}. \tag{2.75}$$

Clearly $V(\mathbf{0}) = 0$ and $V > 0$ for $(\theta, \dot{\theta}) \neq \mathbf{0}$ with $|\theta| < \pi$.
Differentiating along trajectories:

$$\dot{V} = m\ell^2\,\dot{\theta}\,\ddot{\theta} + mg\ell\,\sin\theta\,\dot{\theta} = \dot{\theta}\big(m\ell^2\ddot{\theta} + mg\ell\sin\theta\big) = \dot{\theta}\big(-b\dot{\theta}\big) = -b\,\dot{\theta}^2 \;\leq\; 0. \tag{2.76}$$

Since $\dot{V} \leq 0$, the equilibrium is stable. Because $\dot{V} = 0$ only on the set $\{\dot{\theta} = 0\}$ and no trajectory other than the equilibrium can stay there forever (by LaSalle's invariance principle), the equilibrium is in fact **asymptotically stable**.
Physically, the damper dissipates energy at rate $b\dot{\theta}^2$, and the total energy V is a non-increasing "altitude" that the system rolls down until it reaches the bottom of the energy landscape [**?**].

In Plain Language 2.4. Lyapunov Functions Are Like Altitude

Lyapunov functions are like altitude—if you can prove you are always going downhill, you must eventually reach the valley. The function $V(\mathbf{x})$ measures how "high" the state is above the equilibrium, and the condition $\dot{V} \leq 0$ guarantees that the system never climbs. Finding a good Lyapunov function is an art: energy is often the first candidate, but there are systematic construction methods for certain system classes [**?**].

Mathematical Principle 2.3. Lyapunov Theory as Foundation

Lyapunov stability theory is the foundation of virtually all stability proofs in modern control. Every adaptive controller, every robust controller, and every learning-based stability certificate in Chapters 11–12 ultimately relies on constructing a Lyapunov (or Lyapunov-like) function and showing that its derivative is non-positive along closed-loop trajectories.

2.19 Further Reading

The state-space framework presented in this chapter traces its origins to Kalman's foundational 1960 paper [**?**], which introduced the concepts of controllability and observability that remain central to modern control engineering. That single paper launched an entire field; readers who wish to appreciate its full scope should read the original.

For nonlinear extensions of the state-space ideas developed here, Slotine and Li [**?**] provide an accessible treatment oriented toward robotic systems, covering sliding-mode control, adaptive control, and Lyapunov-based design—all formulated in state space. A more mathematically rigorous treatment of nonlinear systems theory, including stability in the sense of Lyapunov, input-output stability, and singular perturbation methods, can be found in Khalil [**?**].

Spong, Hutchinson, and Vidyasagar [**?**] develop state-space models specifically for robotic manipulators and mobile robots, making explicit the connection between the equations of motion derived from Lagrangian or Newton–Euler methods and the $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ formalism of this chapter.

Finally, readers should note the deep connection between this chapter and Chapter 11 (Lagrangian Mechanics): the Euler–Lagrange equations provide the equations of motion that, once cast in first-order form, populate the state-space models studied here. Understanding both perspectives—the energy-based derivation of dynamics and the state-space representation of those dynamics—is essential for modern robotics.

2.20 Exercises

Computational Exercises

1. For the DC motor from Example 2.3.2, suppose $L = 0.01$ H, $R = 1\ \Omega$, $K_e = 0.02$ V·s/rad, $K_t = 0.02$ N·m/A, $J = 0.001$ kg·m^2, $b = 0.001$ N·m·s/rad. Write the system in the form $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}V_{\text{in}}$. Compute the eigenvalues of $\mathbf{A}$ numerically and classify the open-loop stability.

2. For the two-tank system (Example 2.3.3), assume $A = 0.1$ m^2, $C = 0.05$, $C_{\text{out}} = 0.03$. Starting from $h_1(0) = 0.5$ m, $h_2(0) = 0.2$ m, with constant inflow $Q_{\text{in}} = 0.01$ m^3/s, simulate the system for 100 seconds and plot $h_1(t)$ and $h_2(t)$.

3. Consider the mass-spring-damper with $m = 2$ kg, $c = 4$ N·s/m, $k = 3$ N/m. Compute the damping ratio ζ. Is the system underdamped, critically damped, or overdamped? Plot the phase portrait for initial conditions $(q, \dot{q}) = (1, 0)$ and $(0.5, 1)$.

4. For the pendulum system (Example 2.3.1) with $L = 1$ m, $g = 9.81$ m/s^2, write the nonlinear state equations. Linearize around both the upright ($\theta = 0$) and inverted ($\theta = \pi$) equilibria. Classify each equilibrium.

Conceptual Exercises

5. Explain in your own words why knowledge of position alone is insufficient to specify the state of a system. Use a simple example (e.g., a mass on a spring) to illustrate why velocity must also be included.

6. Why did Rudolf Kalman's 1960 paper on state-space representations represent a breakthrough compared to classical transfer-function methods? What specific limitations of transfer functions does state space overcome?

7. Consider a 3-dimensional system with eigenvalues $\lambda_1 = -2$, $\lambda_2 = 1 + 2i$, $\lambda_3 = 1 - 2i$. Sketch a qualitative phase portrait. What will happen to a trajectory starting near the origin?

8. A system is controllable but not observable. What does this mean physically? Can you still design a feedback controller, and if so, what information would you need?

9. The Kalman rank condition for controllability checks that $\text{rank}([\mathbf{B}, \mathbf{AB}, \ldots, \mathbf{A}^{n-1}\mathbf{B}]) = n$. Why is it sufficient to check only up to $\mathbf{A}^{n-1}\mathbf{B}$ and not higher powers?

Proof-Based Exercises

10. Prove that if $\mathbf{A}$ is diagonalizable with eigenvalues $\lambda_1, \ldots, \lambda_n$, then the matrix exponential is:
$$e^{\mathbf{A}t} = \mathbf{P}\text{diag}(e^{\lambda_1 t}, \ldots, e^{\lambda_n t})\mathbf{P}^{-1} \tag{2.77}$$
where $\mathbf{P}$ is the matrix of eigenvectors.

11. Derive the Jacobian matrix for the pendulum system $\ddot{\theta} + (g/L)\sin(\theta) = \tau$ at the bottom equilibrium ($\theta = 0$). Show that the linearized system is $\ddot{\theta} = -(g/L)\theta$ and classify the equilibrium's stability.

12. Show that for a 2D system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ with $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the eigenvalues are $\lambda = \frac{(a+d) \pm \sqrt{(a-d)^2 + 4bc}}{2}$. Under what conditions are the eigenvalues (i) both real, (ii) complex conjugates?

13. Prove that the controllability matrix $\mathbf{C}_{\text{rank}} = [\mathbf{B}, \mathbf{A}\mathbf{B}, \ldots, \mathbf{A}^{n-1}\mathbf{B}]$ has full rank if and only if there is no eigenvector of $\mathbf{A}$ orthogonal to $\mathbf{B}^T$.

14. Using the variation of constants formula, show that the solution to $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}(t)$ with $\mathbf{x}(0) = \mathbf{x}_0$ is:
$$\mathbf{x}(t) = e^{\mathbf{A}t}\mathbf{x}_0 + \int_0^t e^{\mathbf{A}(t-\tau)}\mathbf{B}\mathbf{u}(\tau)\,\mathrm{d}\tau \tag{2.78}$$

15. (Challenge) Prove that a system is observable (can reconstruct initial state from output) if and only if the observability matrix $\mathbf{O} = [\mathbf{C}^T, \mathbf{A}^T\mathbf{C}^T, \ldots, (\mathbf{A}^T)^{n-1}\mathbf{C}^T]^T$ has full rank. Hint: Use the dual system.

Chapter 3

Configuration Space and Degrees of Freedom

To command a body to move, we must first establish exactly where it is. We do not use the raw atoms of the universe to map a body; we use its independent coordinates—its configuration.

In Plain Language 3.1. Context & Use Case: Mapping the Possible

The Math You Will See: We define the **Configuration Space** ($\mathcal{Q}$), count Degrees of Freedom (DOF) using Grübler's formula, establish the mathematics of **differentiable manifolds**, define **Forward Kinematics** maps, analyze reachable workspace, and distinguish **holonomic** from **non-holonomic** constraints.
Why We Need It: If you want a robot arm to pick up a coffee cup, you know where the cup is in physical 3D space (X, Y, Z). But you cannot command the robot's X, Y, Z position directly! You can only command the motors at its joints (its "configuration"). We must build mathematical maps that translate between the physical space where the coffee cup lives, and the abstract angular "configuration space" where the robot's motors live. The dimension of this space is determined by DOF; its topology and geometry are governed by differential geometry.
All Variables Defined: $\mathbf{q}$ represents joint angles, $\mathcal{Q}$ is the configuration manifold, $\mathbf{p}_{end}$ is the end-effector position, and m, n, c denote the number of bodies, DOF, and independent constraints in Grübler's formula.

3.1 Historical Context: From Lagrange to Riemann

The mathematical framework of Configuration Space owes its development to three foundational ideas.

Lagrange's Generalized Coordinates (1788): Joseph-Louis Lagrange introduced the concept that one need not track every particle's Cartesian coordinates. Instead, for a constrained mechanical system, one could choose a minimal set of independent variables $q_1, q_2, \ldots, q_n$ whose values completely specify the system's configuration. This abstraction—choosing coordinates that *already respect* the system's constraints—transformed mechanics from a coordinate-fixing discipline into a geometric one. The Lagrangian formulation of

mechanics, summarized in the principle of least action, operates naturally in generalized coordinates.

Riemann's Manifold Concept (1854): Bernhard Riemann's Habilitationsvortrag introduced the idea of manifolds: spaces that are locally Euclidean but globally curved. Riemann showed that curvature itself is intrinsic to a space, not imposed by an embedding in a higher-dimensional flat space. This insight enables us to treat Configuration Space not as a rigid subset of $\mathbb{R}^n$, but as an independent geometric object with its own metric and topology.

Modern Differential Geometry: The 20th-century formalization by Élie Cartan, Charles Ehresmann, and others crystallized these ideas into a rigorous calculus on manifolds. Today, Configuration Space is understood as a smooth differentiable manifold, the tangent bundle $T\mathcal{Q}$ forms the state space, and the Jacobian matrix encodes the instantaneous kinematic relationship between joint velocities and end-effector velocities.

3.2 Degrees of Freedom (DOF) and Grübler's Formula

In Chapter 2, we learned that the State Space $\mathcal{X}$ consists of both position coordinates ($\mathbf{q}$) and velocity coordinates ($\dot{\mathbf{q}}$). But how many position coordinates does our system actually have?

3.2.1 The Concept of Degrees of Freedom

Consider a single speck of dust floating completely unconstrained in a room. To perfectly define where it is, you need 3 coordinates: x, y, z. If it is instead a rigid body floating in the room, knowing x, y, z only fixes its center of mass. It could be pitched entirely backward, or rolled on its side. It therefore requires three additional **angles** (roll, pitch, yaw) defining its rotation.

This unconstrained rigid body has exactly 6 **Degrees of Freedom (DOF)**. It requires a minimum of 6 independent parameters (or coordinates) to fully define its posture in 3D Euclidean space.

Definition 3.1. Degrees of Freedom (DOF)

The Degrees of Freedom of a system represents the absolute minimum number of independent scalar coordinates required to perfectly and unambiguously define the position and orientation of every single particle in that system. Equivalently, DOF is the dimension of the Configuration Space $\mathcal{Q}$.

If you connect two floating rigid bodies together with a perfectly rigid bar (welded so they cannot move relative to each other), you do not have 12 DOF. Since the distance and orientation between them are perfectly locked by the weld, you still only have 6 DOF for the combined assembly. The weld acts as a geometric *constraint*. Every independent constraint you add mathematically subtracts one DOF. A hinge joint locking two 6-DOF bodies together removes 5 DOF, leaving exactly 1 DOF of relative rotation between them.

3.2.2 Grübler's Formula

For planar mechanisms, we can count DOF systematically using **Grübler's formula**, developed by Martin Grübler in 1883. This formula gives the degrees of freedom of a kinematic chain.

Theorem 3.1. Grübler's Formula for Planar Mechanisms

For a planar mechanism with m rigid bodies (including ground as one body), n movable links, and c independent geometric constraints, the degrees of freedom are given by:

$$\text{DOF} = 3n - \sum_{i} c_i \tag{3.1}$$

where the sum is over all c independent constraints imposed by joints.
Equivalently, if there are j joints with degrees of freedom $d_1, d_2, \ldots, d_j$:

$$\text{DOF} = 3(n) - \sum_{i=1}^{j}(3 - d_i) = 3n - 3j + \sum_{i=1}^{j} d_i \tag{3.2}$$

Proof. In a planar mechanism, each of n movable bodies independently has 3 DOF (2 translational + 1 rotational). If there were no constraints, the total would be $3n$. Each joint imposes geometric constraints: a revolute joint (pin joint) constrains 2 DOF of relative motion, a prismatic joint constrains 2 DOF, and higher-order joints constrain accordingly. The net DOF equals the free DOF minus the sum of imposed constraint DOF. □

For mechanisms with spatial (3D) linkages, the formula becomes:

Theorem 3.2. Grübler's Formula for Spatial Mechanisms

For a spatial mechanism with n movable links and j joints where joint i permits d_i relative DOF:

$$\text{DOF} = 6n - \sum_{i=1}^{j}(6 - d_i) \tag{3.3}$$

3.2.3 Worked Example 1: Planar 4-Bar Linkage

Example 3.1. Four-Bar Mechanism DOF

Consider the classic planar 4-bar linkage: a mechanism with 4 rigid links (one of which is the ground frame) connected by 4 revolute joints.
We have:

- $n = 3$ movable links (link 1, 2, 3; ground is fixed)
- $j = 4$ revolute joints
- Each revolute joint has $d_i = 1$ (one DOF: rotation about the pin)

Applying Grübler's formula:

$$\text{DOF} = 3n - 3j + \sum_{i=1}^{4} d_i \tag{3.4}$$

$$= 3(3) - 3(4) + 4(1) \tag{3.5}$$

$$= 9 - 12 + 4 = 1 \tag{3.6}$$

A four-bar linkage has exactly **1 DOF**. Once you specify the angle of one input link (the crank), the positions and angles of all other links are completely determined by the geometric constraints. This is why a four-bar can be driven by a single motor to produce a controlled path motion.

3.2.4 Worked Example 2: Spatial Stewart Platform

Example 3.2. Stewart Platform DOF

The Stewart platform (also called a Gough-Stewart platform) is a parallel manipulator consisting of a base platform, a movable top platform, and six variable-length actuators connecting them.
Configuration:

- $n = 1$ movable body (the top platform; the base is ground)
- $j = 12$ joints total: 6 revolute joints at the base (one per leg) and 6 ball joints at the top (one per leg)
- Base revolute joints: each has $d_i = 1$
- Ball joints: each has $d_i = 3$ (allowing 3-DOF rotation)

Actually, for a simpler accounting: the Stewart platform is driven by 6 linear actuators (one per leg), each with 1 DOF (extension/retraction). The constraints are:

- 6 distance constraints from the 6 legs (6 constant lengths)
- Each constraint removes 1 DOF from the platform's 6 spatial DOF

Thus:

$$\text{DOF} = 6 - 6 = 6 \tag{3.7}$$

Wait—this requires careful accounting. The platform has 6 DOF in free space. The 6 leg-length constraints fully determine the platform's pose (position and orientation). If the 6 legs are driven by actuators with specified lengths, then:

$$\text{DOF} = 6 \text{ (input controls)} - 6 \text{ (constraints)} = 6 \text{ (DOF in pose)} \tag{3.8}$$

Actually, the standard result is: a non-degenerate Stewart platform has exactly **6 DOF** in its end-effector pose, and requires 6 actuators to control all 6 spatial DOF. This makes it a fully parallel manipulator.

3.2.5 Worked Example 3: Human Arm Kinematics

Example 3.3. Human Arm DOF

Consider a simplified model of the human arm: shoulder, elbow, and wrist.

- Shoulder: ball-and-socket joint $\approx$ 3 DOF (roll, pitch, yaw about the shoulder center)
- Elbow: hinge joint $\approx$ 1 DOF (flexion/extension)
- Wrist: ball-and-socket joint $\approx$ 3 DOF (pronation/supination, flexion/extension, radial/ulnar deviation)

Total: $3 + 1 + 3 = 7$ DOF in a gross anatomical model. (The human arm actually has 7 DOF, which is why we can reach and reorient objects in multiple ways—we have

redundancy.)
The hand position can be specified by 3 Cartesian coordinates (x, y, z) and the wrist orientation by 3 angles (roll-pitch-yaw). Thus the end-effector pose has 6 DOF. With 7 joint angles, the arm is *redundant*: there are infinitely many joint configurations $\mathbf{q} \in \mathbb{R}^7$ that achieve the same end-effector pose. This redundancy is exploited in human motor control to avoid obstacles and optimize reach.

3.3 The Configuration Space $\mathcal{Q}$ and Topological Manifolds

The collection of all valid configurations a system can possibly achieve—accounting for all of its physical constraints, linkages, pins, and bearings—is called its **Configuration Space**, mathematically denoted by the symbol $\mathcal{Q}$.

Like state space, configuration space is an abstract, multi-dimensional geometric surface. It is, strictly speaking, a mathematical **manifold** [**?**]. The number of dimensions of this space is exactly equal to the system's Degrees of Freedom.

Definition 3.2. Topological Manifold

A *topological manifold* of dimension n is a topological space $\mathcal{M}$ such that:

1. $\mathcal{M}$ is Hausdorff (any two distinct points have disjoint open neighborhoods),
2. $\mathcal{M}$ is second-countable (has a countable basis for its topology),
3. Every point $p \in \mathcal{M}$ has an open neighborhood U such that U is homeomorphic to an open subset of $\mathbb{R}^n$.

In less formal terms: a manifold *locally looks like Euclidean space*, but globally may have a curved or non-trivial topology.

3.3.1 Examples of Manifolds

Example 3.4. The Circle S^1

The circle $S^1 = \{(x, y) \in \mathbb{R}^2 : x^2 + y^2 = 1\}$ is a 1-dimensional manifold. Globally, it closes on itself and is topologically distinct from a line. However, locally, any small arc of the circle looks like a line segment from $\mathbb{R}^1$.

Example 3.5. The 2-Sphere S^2

The 2-sphere $S^2 = \{(x, y, z) \in \mathbb{R}^3 : x^2 + y^2 + z^2 = 1\}$ is a 2-dimensional manifold (the Earth's surface). Globally, it is closed and without boundary. Locally, near any point (except the poles in certain coordinates), it looks like a flat 2D plane.

Example 3.6. The Torus T^2

The torus $T^2 = S^1 \times S^1$ is the Cartesian product of two circles. It is a 2-dimensional manifold. Topologically, it can be visualized as the surface of a doughnut: closed without boundary, genus 1.

In Plain Language 3.2. Configuration Space vs. State Space

A system with n Degrees of Freedom has a Configuration Space $\mathcal{Q}$ of dimension n, defined completely by the vector $\mathbf{q} = [q_1, q_2, \dots, q_n]^T$.
The corresponding **State Space** $\mathcal{X}$ (from Chapter 2) is the configuration space multiplied by the velocity of those coordinates, meaning state space always has $2n$ dimensions.

$$\text{State } \mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T \in \mathcal{X} = T\mathcal{Q}$$

(Where $T\mathcal{Q}$ represents the Tangent Bundle of the configuration manifold).

3.4 Charts, Atlases, and Coordinate Patches

While the configuration manifold $\mathcal{Q}$ is curved globally, our primary computational tool is always Euclidean coordinates. We use local maps called *charts* to cover the manifold piecewise with flat coordinate systems.

Definition 3.3. Chart

A *chart* on a manifold $\mathcal{Q}$ is a pair (U, φ) where:

- $U \subseteq \mathcal{Q}$ is an open set,
- $\varphi : U \to \mathbb{R}^n$ is a homeomorphism (continuous bijection with continuous inverse) onto an open subset of $\mathbb{R}^n$.

The map φ assigns local coordinates (real numbers) to each point in U.

Definition 3.4. Atlas

An *atlas* on a manifold $\mathcal{Q}$ is a collection of charts $\{(U_i, \varphi_i)\}_{i \in I}$ such that:

- The sets U_i cover the entire manifold: $\mathcal{Q} = \bigcup_{i \in I} U_i$,
- The charts are pairwise compatible in a sense we clarify below.

3.4.1 Worked Example: Charts on the Circle S^1

Example 3.7. Two Charts Are Necessary for S^1

Consider the unit circle $S^1 = \{(x, y) : x^2 + y^2 = 1\}$. Can we use a single chart $\varphi : S^1 \to \mathbb{R}$ that maps every point on the circle to a real number?

Attempt 1: Naive Angle Parameterization

Define $\varphi(\theta) = \theta$ where $\theta \in [0, 2\pi)$ is the angle from the positive x-axis. This gives a bijection between most of S^1 and $[0, 2\pi)$. However, this map has two problems:

- The interval $[0, 2\pi)$ is not homeomorphic to $\mathbb{R}$ (it is half-open and bounded).
- The map is not continuous at the boundary where $\theta = 0$ and $\theta = 2\pi$ identify the same point.

A single chart cannot cover all of S^1 continuously.

Solution: Two Overlapping Charts

Define two open sets:

$$U_1 = \{(x, y) \in S^1 : y > -\epsilon\} \quad \text{(excludes a small arc at the bottom)} \tag{3.9}$$

$$U_2 = \{(x, y) \in S^1 : y < \epsilon\} \quad \text{(excludes a small arc at the top)} \tag{3.10}$$

where ϵ is small.

On U_1, use the chart (U_1, φ_1) where $\varphi_1(x, y) = x$. Since $y > -\epsilon$, the x-coordinate alone determines the point (by the Pythagorean theorem, $x \in (-1, 1)$), and φ_1 maps homeomorphically onto the interval $(-1, 1) \subset \mathbb{R}$.

On U_2, use the chart (U_2, φ_2) where $\varphi_2(x, y) = -x$. This also maps homeomorphically onto $(-1, 1)$.

The two charts overlap on $U_1 \cap U_2 = \{(x, y) \in S^1 : -\epsilon < y < \epsilon\}$. This overlap is essential: we have covered the entire circle with two overlapping charts, each mapping to Euclidean space $\mathbb{R}$.

Key Insight: The necessity of two charts reflects the topological fact that the circle is not contractible: you cannot continuously shrink it to a point. This global topology cannot be captured by a single local coordinate system.

3.5 Transition Maps and Smooth Compatibility

When two charts overlap, we must ensure they agree on their shared region. This is formalized by *transition maps*.

Definition 3.5. Transition Map

If (U_i, φ_i) and (U_j, φ_j) are two charts whose domains overlap, the *transition map* is defined as:

$$\tau_{ij} = \varphi_j \circ \varphi_i^{-1} : \varphi_i(U_i \cap U_j) \to \varphi_j(U_i \cap U_j) \tag{3.11}$$

This map takes coordinates from one chart and converts them to coordinates in another.

For a *smooth* (or *differentiable*) manifold, we require all transition maps to be smooth (infinitely differentiable).

Definition 3.6. Differentiable Manifold

A differentiable manifold is a topological manifold equipped with a differentiable structure: an atlas where every transition map is C^∞ (infinitely differentiable).

3.5.1 Worked Example: Transition Map on S^1

Example 3.8. Transition Map on the Circle

Using the two charts from Example 3.4.1:

- Chart 1: (U_1, φ_1) where $\varphi_1(x, y) = x$ for $y > -\epsilon$.
- Chart 2: (U_2, φ_2) where $\varphi_2(x, y) = -x$ for $y < \epsilon$.

On the overlap $U_1 \cap U_2$, we have points (x, y) with $-\epsilon < y < \epsilon$.
In chart 1 coordinates, a point is labeled by $x_1 = x \in (-1, 1)$.
In chart 2 coordinates, the same point is labeled by $x_2 = -x \in (-1, 1)$.
The transition map is:

$$\tau_{12}(x_1) = -x_1 \tag{3.12}$$

Equivalently, going the other direction:

$$\tau_{21}(x_2) = -x_2 \tag{3.13}$$

Both are clearly C^∞ (linear functions, hence infinitely differentiable). This smoothness ensures that the calculus we do with one chart's coordinates yields the same results when computed in another chart's coordinates.

3.6 Tangent Vectors, Curves, and Derivations

While the manifold $\mathcal{Q}$ represents all valid positions (configurations), motion requires us to talk about velocities. Because the manifold is curved, a velocity vector cannot "live" on the surface itself—it would point off the surface tangentially!

Definition 3.7. Tangent Space $T_q\mathcal{Q}$

At any specific configuration $q \in \mathcal{Q}$, the *Tangent Space* $T_q\mathcal{Q}$ is a vector space whose elements are called *tangent vectors*. It is the space of all possible velocities at q.

There are two rigorous ways to define tangent vectors: as equivalence classes of curves, and as derivations on function algebras. Both are equivalent, and understanding both perspectives

is essential.

3.6.1 Tangent Vectors as Equivalence Classes of Curves

Definition 3.8. Tangent Vector via Curves

Let $q \in \mathcal{Q}$ be a point on the manifold. Two smooth curves $\gamma_1, \gamma_2 : (-\epsilon, \epsilon) \to \mathcal{Q}$ with $\gamma_1(0) = \gamma_2(0) = q$ are said to be *tangent at q in a chart (U, φ)* if:

$$\left.\frac{d}{dt}\right|_{t=0} \varphi(\gamma_1(t)) = \left.\frac{d}{dt}\right|_{t=0} \varphi(\gamma_2(t)) \tag{3.14}$$

(The derivatives of their coordinate expressions agree at $t = 0$.)
A *tangent vector* at q is an equivalence class $[\gamma]$ of curves all tangent to each other at q. The value $\mathbf{v} = \frac{d}{dt}\big|_{t=0} \varphi(\gamma(t)) \in \mathbb{R}^n$ is the coordinate representation of the tangent vector in the chart.

This definition is independent of the chart chosen: if two tangent vectors agree in one chart's coordinates, they agree in all charts' coordinates (by properties of transition maps).

3.6.2 Tangent Vectors as Derivations

An alternative algebraic viewpoint: a tangent vector is a linear operator on smooth functions.

Definition 3.9. Tangent Vector via Derivations

A *derivation at q* is a linear map $v : C^\infty(\mathcal{Q}) \to \mathbb{R}$ (where $C^\infty(\mathcal{Q})$ is the algebra of smooth real-valued functions on $\mathcal{Q}$) such that:

$$v(fg) = v(f)g(q) + f(q)v(g) \quad \text{(Leibniz rule)} \tag{3.15}$$

A *tangent vector* at q can equivalently be defined as a derivation at q.

The Leibniz rule (or product rule) encodes the essential algebraic property of derivatives. A tangent vector is "an infinitesimal displacement" in the sense that it measures the first-order change in functions along infinitesimal directions.

3.6.3 Equivalence of the Two Definitions

Theorem 3.3. Equivalence of Tangent Vector Definitions

The space of equivalence classes of curves passing through q (Definition 3.8) is isomorphic, as a vector space, to the space of derivations at q (Definition 3.9). Both have dimension n (the dimension of $\mathcal{Q}$).

Proof. Given a curve γ with $\gamma(0) = q$, define a derivation by:

$$v_\gamma(f) = \left.\frac{d}{dt}\right|_{t=0} f(\gamma(t)) \tag{3.16}$$

This satisfies the Leibniz rule by the product rule of calculus. Conversely, given a derivation v, one can construct curves tangent to that direction. The maps between these spaces are linear isomorphisms, preserving dimension. □

The intuition: curves give a geometric picture of infinitesimal motion; derivations give an algebraic picture of how functions change. Both capture the same mathematical object.

3.6.4 The Tangent Bundle $T\mathcal{Q}$

Definition 3.10. Tangent Bundle

The *Tangent Bundle* $T\mathcal{Q}$ is the disjoint union of all tangent spaces:

$$T\mathcal{Q} = \bigcup_{q \in \mathcal{Q}} T_q\mathcal{Q} \tag{3.17}$$

Equipped with a natural smooth structure, $T\mathcal{Q}$ is itself a differentiable manifold of dimension $2n$ (if $\mathcal{Q}$ has dimension n).

An element of $T\mathcal{Q}$ is a pair $(q, \mathbf{v})$ where $q \in \mathcal{Q}$ is a configuration and $\mathbf{v} \in T_q\mathcal{Q}$ is a velocity at that configuration.

In Plain Language 3.3. Mathematical Reminder: Tangent Bundle vs. State Space

Whenever you see the State Vector $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$, recall its geometric meaning. The upper half ($\mathbf{q}$) is a point on the curved manifold $\mathcal{Q}$. The lower half ($\dot{\mathbf{q}}$) is a vector living in the flat tangent space $T_q\mathcal{Q}$ attached specifically to that point $\mathbf{q}$. Together, they coordinatize the Tangent Bundle $T\mathcal{Q}$, which is exactly the State Space $\mathcal{X}$.

3.7 Generalized Coordinates and Forward Kinematics

When we model a complex physical system (such as the 15-DOF human golfer in Volume III), we do not track the 3D (x, y, z) position of the golfer's wrist, elbow, and shoulder separately, and then painstakingly write huge algebraic constraint equations ensuring their bones don't stretch or compress during simulation.

Instead, we use **Generalized Coordinates**. Since an elbow is a pin joint allowing only 1 DOF of rotation, we abandon (x, y, z) mapping entirely and instead just track the single angle θ_{elbow}. If we define the angle of the shoulder θ_{shoulder}, and the fixed length of the humerus bone L_1, the Cartesian position of the elbow is instantly known.

Definition 3.11. Generalized Coordinates

Generalized coordinates are the minimal set of independent variables $q_1, q_2, \ldots, q_n$ that perfectly span the Configuration Space, such that every point in $\mathcal{Q}$ is uniquely represented by some choice of $(q_1, \ldots, q_n)$. They are the most mathematically efficient map of the mechanism's geometry. In robotics, $\mathbf{q}$ almost always refers to the physical joint

angles of the motors.

3.7.1 Forward Kinematics

The mathematical function that translates generalized coordinates $\mathbf{q}$ back into the physical 3D Euclidean space of the real world is called **Forward Kinematics**.

Definition 3.12. Forward Kinematics Map

If $\mathbf{p}_{end} = [x, y, z]^T \in \mathbb{R}^3$ is the Cartesian workspace position of the end-effector of a robotic arm, Forward Kinematics is the generally nonlinear function $f_{kin} : \mathcal{Q} \to \mathbb{R}^3$ defined by:

$$\mathbf{p}_{end} = f_{kin}(\mathbf{q}) \tag{3.18}$$

By using trigonometry and the chain rule of coordinate composition, we stack the lengths of the links and the angles of the joints $\mathbf{q}$ to compute exactly where the end-effector sits in physical space.

Example: 2-Link Planar Arm

For a simple 2-link robotic arm rotating on a flat 2D plane with fixed link lengths L_1, L_2 and variable joint angles q_1, q_2:

$$x = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) \tag{3.19}$$
$$y = L_1 \sin(q_1) + L_2 \sin(q_1 + q_2) \tag{3.20}$$

The forward kinematics map is $f_{kin}(q_1, q_2) = [x(q_1, q_2), y(q_1, q_2)]^T$.

3.8 The Jacobian Matrix and the Kinematic Derivative

To study the instantaneous kinematic relationship between joint velocities $\dot{\mathbf{q}}$ and end-effector velocities $\dot{\mathbf{p}}$, we differentiate the forward kinematics [**?**].

Definition 3.13. Jacobian Matrix of Forward Kinematics

The Jacobian matrix of $f_{kin} : \mathcal{Q} \to \mathbb{R}^m$ at a configuration $\mathbf{q}$ is:

$$J_{kin}(\mathbf{q}) = \frac{\partial f_{kin}}{\partial \mathbf{q}}\bigg|_{\mathbf{q}} = \begin{bmatrix} \frac{\partial f_1}{\partial q_1} & \cdots & \frac{\partial f_1}{\partial q_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial q_n} & \cdots & \frac{\partial f_m}{\partial q_n} \end{bmatrix} \tag{3.21}$$

where m is the dimension of the workspace (usually $m = 3$ for position in 3D space, or $m = 6$ if orientation is included).

By the chain rule, the end-effector velocity is related to joint velocity by:

$$\dot{\mathbf{p}} = J_{kin}(\mathbf{q})\dot{\mathbf{q}} \tag{3.22}$$

3.8.1 Worked Example: 2-Link Arm Jacobian

Example 3.9. Jacobian of 2-Link Planar Arm

For the 2-link planar arm from Section 3.7:

$$x(q_1, q_2) = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) \tag{3.23}$$
$$y(q_1, q_2) = L_1 \sin(q_1) + L_2 \sin(q_1 + q_2) \tag{3.24}$$

Compute the partial derivatives:

$$\frac{\partial x}{\partial q_1} = -L_1 \sin(q_1) - L_2 \sin(q_1 + q_2) \tag{3.25}$$
$$\frac{\partial x}{\partial q_2} = -L_2 \sin(q_1 + q_2) \tag{3.26}$$
$$\frac{\partial y}{\partial q_1} = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) \tag{3.27}$$
$$\frac{\partial y}{\partial q_2} = L_2 \cos(q_1 + q_2) \tag{3.28}$$

Thus, the Jacobian is:

$$J_{kin}(q_1, q_2) = \begin{bmatrix} -L_1 \sin(q_1) - L_2 \sin(q_1 + q_2) & -L_2 \sin(q_1 + q_2) \\ L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) & L_2 \cos(q_1 + q_2) \end{bmatrix} \tag{3.29}$$

This is a 2×2 matrix (workspace dimension 2, joint DOF 2). For any joint velocity $[\dot{q}_1, \dot{q}_2]^T$, the end-effector velocity is:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = J_{kin} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \tag{3.30}$$

The Jacobian depends on the configuration: it changes as the arm moves through $\mathcal{Q}$.

3.8.2 Worked Example: 3-Link Arm in 3D

Example 3.10. Jacobian of 3-Link Arm with Orientation

Consider a 3-link planar arm (same plane), but now we also track the orientation of the end-effector (the angle of the end link).
Forward kinematics:

$$x = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) + L_3 \cos(q_1 + q_2 + q_3) \tag{3.31}$$
$$y = L_1 \sin(q_1) + L_2 \sin(q_1 + q_2) + L_3 \sin(q_1 + q_2 + q_3) \tag{3.32}$$
$$\theta = q_1 + q_2 + q_3 \tag{3.33}$$

The task space is now 3-dimensional: (x, y, θ). The Jacobian is 3×3:

$$J_{kin} = \begin{bmatrix} -L_1\sin(q_1) - L_2\sin(q_1+q_2) - L_3\sin(q_1+q_2+q_3) & -L_2\sin(q_1+q_2) - L_3\sin(q_1+q_2+q_3) & -L_3 \text{s} \\ L_1\cos(q_1) + L_2\cos(q_1+q_2) + L_3\cos(q_1+q_2+q_3) & L_2\cos(q_1+q_2) + L_3\cos(q_1+q_2+q_3) & L_3 \text{co} \\ 1 & 1 & \end{bmatrix} \tag{3.34}$$

When the Jacobian is square and full rank, the arm is in a non-singular configuration: there is a unique relationship between joint velocities and task velocities. When the determinant is zero, the arm enters a singularity, and some task velocities become unreachable.

3.9 Inverse Kinematics

The forward kinematics problem is: given joint angles $\mathbf{q}$, find the end-effector position $\mathbf{p} = f_{kin}(\mathbf{q})$.

The inverse problem is fundamentally harder: given a desired end-effector position $\mathbf{p}_d$, find the joint angles $\mathbf{q}$ such that $f_{kin}(\mathbf{q}) = \mathbf{p}_d$.

Definition 3.14. Inverse Kinematics Problem

The Inverse Kinematics problem is: solve for $\mathbf{q}$ given $\mathbf{p}_d$:

$$f_{kin}(\mathbf{q}) = \mathbf{p}_d \tag{3.35}$$

Inverse kinematics is harder for three reasons:

1. *Non-uniqueness:* Multiple joint configurations may reach the same end-effector position. For a 2-link arm, you can reach a target by bending the elbow forward (elbow-up) or backward (elbow-down).
2. *Non-existence:* Some end-effector positions may be unreachable (outside the workspace).
3. *Nonlinearity:* The forward kinematics is nonlinear (contains sines and cosines), making the inverse algebraically intractable in general.

3.9.1 Geometric Approach to 2-Link IK

For the 2-link planar arm, we can solve geometrically.

Example 3.11. 2-Link Arm Inverse Kinematics (Geometric)

Given $\mathbf{p}_d = [x_d, y_d]^T$ and link lengths L_1, L_2, find $[q_1, q_2]^T$.

Step 1: Use the law of cosines to find the distance $r = \sqrt{x_d^2 + y_d^2}$ from the base to the target. The angle q_2 (elbow angle) satisfies:

$$r^2 = L_1^2 + L_2^2 - 2L_1L_2\cos(\pi - q_2) \tag{3.36}$$

(Here $\pi - q_2$ is the interior angle at the elbow.)

Solving:

$$\cos(q_2) = \frac{L_1^2 + L_2^2 - r^2}{2L_1 L_2} \tag{3.37}$$

Step 2: Once q_2 is known, the elbow position is constrained. The shoulder angle q_1 can be found from:

$$q_1 = \text{atan2}(y_d, x_d) - \text{atan2}(L_2 \sin(q_2), L_1 + L_2 \cos(q_2)) \tag{3.38}$$

This geometric approach yields, generically, two solutions (elbow-up and elbow-down), corresponding to the two square roots in solving the cosine equation.

3.9.2 Numerical Inverse Kinematics: Newton-Raphson

For higher DOF arms or when geometric solutions don't exist, we use numerical optimization.

Theorem 3.4. Newton-Raphson for Inverse Kinematics

Define the error function:

$$\mathbf{e}(\mathbf{q}) = \mathbf{p}_d - f_{kin}(\mathbf{q}) \tag{3.39}$$

The Newton-Raphson iteration is:

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \alpha_k J_{kin}(\mathbf{q}_k)^\dagger \mathbf{e}(\mathbf{q}_k) \tag{3.40}$$

where $J_{kin}^\dagger$ is the pseudo-inverse of the Jacobian and α_k is a step size (often $\alpha_k = 1$ initially, or line search if convergence is slow).
When J_{kin} is square and full rank, $J_{kin}^\dagger = J_{kin}^{-1}$.

This iterative method starts from an initial guess $\mathbf{q}_0$ and updates it repeatedly until $\|\mathbf{e}(\mathbf{q}_k)\|$ is small. Convergence is fast (quadratic) near a solution, but global convergence is not guaranteed.

3.10 Workspace Analysis: Reachable and Dexterous Workspace

Once we have a forward kinematics function, a key question is: what end-effector positions can the robot actually reach?

Definition 3.15. Reachable Workspace

The *reachable workspace* of a robot arm is the set of all end-effector positions $\mathbf{p}$ that can be reached by at least one configuration $\mathbf{q}$:

$$\mathcal{W}_{reach} = \{\mathbf{p} \in \mathbb{R}^m : \exists \mathbf{q} \in \mathcal{Q}, f_{kin}(\mathbf{q}) = \mathbf{p}\} \tag{3.41}$$

It is the image of f_{kin}.

Definition 3.16. Dexterous Workspace

The *dexterous workspace* is the subset of reachable workspace where the robot can orient the end-effector in all directions (or at least a specified set of directions):

$$\mathcal{W}_{dex} = \{\mathbf{p} : \exists \mathbf{q} \text{ reaching } \mathbf{p} \text{ with } \det(J_{kin}(\mathbf{q})) \neq 0\} \tag{3.42}$$

(Requires the Jacobian to be full rank in at least one configuration.)

For a 2-link arm with $L_1 = L_2 = 1$:

- The reachable workspace is an annulus (ring) with inner radius $|L_1 - L_2| = 0$ and outer radius $L_1 + L_2 = 2$. Any point (x, y) with $0 \leq \sqrt{x^2 + y^2} \leq 2$ can be reached.
- The dexterous workspace is smaller: a subset of the annulus where configurations exist with full-rank Jacobian.

3.11 The Frobenius Theorem and Integrability

A central question in constraint analysis is: *when does a velocity constraint actually reduce the reachable configuration space?* The answer lies in the **Frobenius Theorem**, which provides an algebraic test for *integrability* of a set of velocity constraints.

Definition 3.17. Smooth Distribution

A *smooth distribution* Δ on a manifold $\mathcal{Q}$ assigns to each point $\mathbf{q} \in \mathcal{Q}$ a linear subspace $\Delta(\mathbf{q}) \subseteq T_{\mathbf{q}}\mathcal{Q}$ of the tangent space. If $\dim \Delta(\mathbf{q}) = k$ for all $\mathbf{q}$, the distribution has *rank* k. A distribution is *integrable* if there exists a foliation of $\mathcal{Q}$ into k-dimensional submanifolds (leaves) whose tangent spaces coincide with Δ.

Intuitively, a distribution collects the "allowed velocity directions" at every configuration. If the distribution is integrable, those allowed velocities confine the system to a lower-dimensional surface—the constraint genuinely eliminates degrees of freedom. If the distribution is *not* integrable, the system can eventually reach any configuration despite the instantaneous velocity restriction.

Theorem 3.5. Frobenius Theorem

A smooth distribution Δ is integrable if and only if it is *involutive*: for every pair of smooth vector fields X, Y belonging to Δ, their Lie bracket also belongs to Δ:

$$X, Y \in \Delta \quad \Longrightarrow \quad [X, Y] \in \Delta. \tag{3.43}$$

When the Frobenius condition *fails*—that is, when there exist vector fields $X, Y \in \Delta$ whose Lie bracket $[X, Y] \notin \Delta$—the distribution is non-integrable. In mechanical terms, this means the velocity constraint is genuinely **non-holonomic**: it restricts instantaneous motion but does *not* reduce the reachable set of configurations.

Example 3.12. The Car: A Non-Integrable Constraint

Consider a car on the plane with configuration $\mathbf{q} = (x, y, \theta) \in \mathbb{R}^2 \times S^1$. The no-lateral-slip constraint is:

$$\dot{y}\cos\theta - \dot{x}\sin\theta = 0. \tag{3.44}$$

This defines a rank-2 distribution Δ spanned by the vector fields:

$$X_1 = \cos\theta\,\frac{\partial}{\partial x} + \sin\theta\,\frac{\partial}{\partial y}, \qquad X_2 = \frac{\partial}{\partial \theta}. \tag{3.45}$$

Computing the Lie bracket:

$$[X_1, X_2] = -\sin\theta\,\frac{\partial}{\partial x} + \cos\theta\,\frac{\partial}{\partial y}. \tag{3.46}$$

Since $[X_1, X_2]$ is *not* a linear combination of X_1 and X_2, the Frobenius condition fails. Therefore the constraint is **non-integrable**: despite the velocity restriction, the car can reach *any* configuration (x, y, θ) in $\mathbb{R}^2 \times S^1$.

Example 3.13. A Rigid Rod: An Integrable Constraint

Consider two particles in $\mathbb{R}^3$ connected by a rigid rod of length L. The constraint $\|\mathbf{r}_1 - \mathbf{r}_2\|^2 = L^2$ depends only on positions (not velocities). Differentiating yields a velocity constraint, but this velocity constraint *is* integrable—the original position-level equation defines a submanifold of the full configuration space. The Frobenius condition is satisfied, and the constraint permanently reduces the DOF from 6 to 5.

In Plain Language 3.4. Parallel Parking and Reachability

A car can parallel park into any spot despite only being able to drive forward/backward and turn—non-holonomic constraints restrict velocity but not reachability. The Frobenius theorem makes this precise: because the car's velocity constraint is non-integrable, repeated manoeuvres (sequences of steering and driving) generate motion in *every* direction of configuration space. This is why parallel parking works, even though you cannot slide the car sideways.

The Frobenius theorem thus provides the rigorous dividing line between constraints that reduce configuration space dimension (holonomic, integrable) and those that merely shape the geometry of feasible paths (non-holonomic, non-integrable). For a comprehensive treatment, see [**?**] Chapter 8 and [**?**].

3.12 Holonomic and Non-Holonomic Constraints

It is critical to distinguish between constraints that restrict position versus those that restrict velocity.

Definition 3.18. Holonomic Constraint

A *holonomic constraint* is a geometric constraint that restricts the actual position of the system. It can be expressed as an equation $\phi(\mathbf{q}) = 0$ involving only the generalized coordinates (not their derivatives). Holonomic constraints permanently reduce the dimension of the Configuration Space.

Example: A train locked to a physical track cannot deviate sideways; if $x(t)$ is its position along the track and $y(t)$ is its lateral deviation, the constraint $y(t) = 0$ for all time is holonomic. This is a geometric constraint on position.

Definition 3.19. Non-Holonomic Constraint

A *non-holonomic constraint* is a constraint on the system's velocity but not its position. It cannot be integrated to a geometric constraint. Non-holonomic constraints do not reduce the dimension of the Configuration Space, but they severely restrict instantaneous motion.

Example: A car's wheels have a no-slip condition: the car cannot move purely sideways. If (X, Y, Θ) are the car's position and heading, the constraint is:

$$\dot{X}\sin(\Theta) - \dot{Y}\cos(\Theta) = 0 \tag{3.47}$$

This is a differential constraint on velocities. However, by parallel parking (going forward, turning, reversing), a car can eventually reach any (X, Y, Θ) on the plane. The Configuration Space remains 3-dimensional, but instantaneous motion is constrained to 1 DOF (forward speed, with heading changing as a function of steering angle).

3.12.1 More Examples of Non-Holonomic Systems

Example 3.14. Rolling Coin Without Slipping

A coin rolling without slipping on a table has constraints:

- Position: (X, Y, Θ) (center location and heading)
- Rotation: ϕ (angle rotated)

No-slip condition: the coin's rotation is proportional to distance traveled:

$$\frac{d\phi}{dt} = \frac{R}{r}\sqrt{\dot{X}^2 + \dot{Y}^2} \tag{3.48}$$

where R is the coin's radius and r is some reference length.
This is a velocity constraint; it does not reduce the position configuration space. But it is very restrictive: the coin cannot instantaneously move sideways while rotating.

Example 3.15. Unicycle Model

A unicycle has configuration (X, Y, Θ) and one input (pedal). The kinematics are:

$$\dot{X} = v\cos(\Theta) \tag{3.49}$$

$$\dot{Y} = v\sin(\Theta) \tag{3.50}$$

$$\dot{\Theta} = \frac{v}{L}\tan(\delta) \tag{3.51}$$

where v is velocity, L is wheelbase, and δ is steering angle.
The non-holonomic constraint is $\dot{X}\sin(\Theta) - \dot{Y}\cos(\Theta) = 0$. The system has 3 DOF position space but only 2 inputs (v, δ), yielding a 2-dimensional input-reachable set in velocity space at each configuration.

Example 3.16. Ball on a Tilted Plate

A ball rolling (without slipping) on an inclined or moving rigid plate in 3D. The ball's position is (x, y) and orientation is a rotation matrix $R \in SO(3)$.
The no-slip constraint couples the ball's linear velocity to its angular velocity:

$$\dot{\mathbf{r}} = \omega \times \mathbf{c} \tag{3.52}$$

where $\mathbf{c}$ is the contact point and ω is the angular velocity.
This is again a velocity constraint. The 5-dimensional position space (3 for position, 3 for orientation minus 1 for the fixed contact constraint) is not reduced to lower dimension, but instantaneous motion is restricted to a lower-dimensional subset.

3.12.2 Pfaffian Constraints

Non-holonomic constraints are often written in Pfaffian form.

Definition 3.20. Pfaffian Constraint

A *Pfaffian constraint* is a linear constraint on velocity of the form:

$$\mathbf{A}(\mathbf{q})d\mathbf{q} = \mathbf{0} \tag{3.53}$$

or equivalently:

$$\mathbf{A}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0} \tag{3.54}$$

where $\mathbf{A}(\mathbf{q}) \in \mathbb{R}^{k\times n}$ is a matrix of constraint coefficients (typically $k < n$), and the rows are assumed to be linearly independent.

Interpretation: The constraint eliminates k directions of velocity from the tangent space. The system can only move in directions orthogonal to the rows of $\mathbf{A}$.

Example 3.17. Car Non-Holonomic Constraint in Pfaffian Form

For a car with configuration $\mathbf{q} = [X, Y, \Theta]^T$, the no-slip constraint is:

$$\dot{X}\sin(\Theta) - \dot{Y}\cos(\Theta) = 0 \tag{3.55}$$

In Pfaffian form:

$$\begin{bmatrix}\sin(\Theta) & -\cos(\Theta) & 0\end{bmatrix}\begin{bmatrix}\dot{X}\\ \dot{Y}\\ \dot{\Theta}\end{bmatrix} = 0 \tag{3.56}$$

This single row eliminates one velocity direction. The car can move in any direction tangent to the constraint (i.e., perpendicular to $[\sin(\Theta), -\cos(\Theta), 0]^T$).

3.13 Python Worked Example: Forward Kinematics and Workspace of a 2R Arm

Example 3.18. Computing Workspace of 2-Link Arm

Below is a complete Python script that computes and visualizes the workspace of a 2-link planar robot arm.

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
L1, L2 = 1.0, 1.0  # link lengths
n_samples = 100    # samples per joint angle

# Create a grid of joint angles
q1 = np.linspace(0, 2*np.pi, n_samples)
q2 = np.linspace(0, 2*np.pi, n_samples)
Q1, Q2 = np.meshgrid(q1, q2)

# Forward kinematics
X = L1 * np.cos(Q1) + L2 * np.cos(Q1 + Q2)
Y = L1 * np.sin(Q1) + L2 * np.sin(Q1 + Q2)

# Plot workspace
plt.figure(figsize=(10, 5))

# Left: scatter plot of all reachable points
plt.subplot(1, 2, 1)
plt.scatter(X, Y, s=1, alpha=0.5, c='blue')
plt.axis('equal')
```

```
plt.xlabel('x')
plt.ylabel('y')
plt.title('Reachable Workspace (2R Arm)')
plt.grid(True)

# Right: compute Jacobian determinant at each configuration
# J = [[-L1*sin(q1) - L2*sin(q1+q2), -L2*sin(q1+q2)]
#      [ L1*cos(q1) + L2*cos(q1+q2),  L2*cos(q1+q2)]]
J11 = -L1*np.sin(Q1) - L2*np.sin(Q1+Q2)
J12 = -L2*np.sin(Q1+Q2)
J21 = L1*np.cos(Q1) + L2*np.cos(Q1+Q2)
J22 = L2*np.cos(Q1+Q2)

det_J = J11*J22 - J12*J21

# Plot dexterous workspace (where det(J) != 0)
plt.subplot(1, 2, 2)
# Color points by Jacobian determinant
plt.scatter(X, Y, s=1, c=np.abs(det_J), cmap='viridis', alpha=0.6)
plt.colorbar(label='|det(J)|')
plt.axis('equal')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Workspace colored by Singularity Distance')
plt.grid(True)

plt.tight_layout()
plt.show()

# Demonstrate forward kinematics at a specific configuration
q = np.array([np.pi/4, np.pi/3])  # example configuration
p = np.array([L1*np.cos(q[0]) + L2*np.cos(q[0]+q[1]),
              L1*np.sin(q[0]) + L2*np.sin(q[0]+q[1])])
print(f"Configuration q = {q}")
print(f"End-effector position p = {p}")
```

This script:

1. Defines link lengths $L_1 = L_2 = 1$.
2. Evaluates forward kinematics at $100 \times 100 = 10{,}000$ configurations in the joint space $[0, 2\pi]^2$.
3. Scatters the resulting end-effector positions to visualize the reachable workspace (should be an annulus).

4. Computes the Jacobian determinant at each configuration and colors points by singularity distance (higher determinant = further from singularities = more dexterous).

5. Demonstrates kinematics at a specific configuration.

The output shows that the 2-link arm can reach any point in the annulus $0 \leq r \leq L_1 + L_2 = 2$, and the dexterous workspace (colored region) is concentrated in the interior where the Jacobian is well-conditioned.

3.14 Chapter Summary and Key Principles

Mathematical Principle 3.1. Configuration Space Essentials

Core Concepts:

- The **Configuration Space** $\mathcal{Q}$ is a manifold of dimension equal to the system's DOF. Every point represents a unique configuration.
- **Degrees of Freedom** are counted via Grübler's formula and represent the minimal number of independent coordinates needed.
- **Generalized coordinates** $\mathbf{q}$ span $\mathcal{Q}$ efficiently, avoiding explicit constraint equations.
- **Forward Kinematics** $\mathbf{p} = f_{kin}(\mathbf{q})$ maps configurations to workspace positions.
- The **Jacobian** $J_{kin} = \frac{\partial f_{kin}}{\partial \mathbf{q}}$ relates joint and task-space velocities.
- The **Tangent Bundle** $T\mathcal{Q}$ (pairs of configurations and velocities) is the state space.

Practical Insights:

- **Charts and atlases** provide local Euclidean coordinates for curved manifolds.
- **Inverse Kinematics** is generally nonlinear and may have multiple (or no) solutions.
- **Workspace** divides into reachable and dexterous regions; singularities mark the boundary.
- **Holonomic constraints** reduce dimension; **non-holonomic constraints** restrict velocity but not position.
- Pfaffian constraints encode velocity restrictions algebraically: $\mathbf{A}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0}$.

3.15 Further Reading

The configuration-space perspective developed in this chapter has its roots in differential geometry. Chapters 2–3 of Murray, Li, and Sastry [?] provide a rigorous treatment of configuration manifolds, tangent spaces, and the kinematic maps that connect joint space to task space, all within the robotics context. Their development of Grübler's formula and the topology of common joint types directly complements the material presented here.

Lynch and Park [?] offer a modern, self-contained treatment of configuration-space analysis with extensive worked examples for serial and parallel mechanisms. Their textbook is particularly strong on workspace computation and kinematic singularities.

For Jacobian analysis, velocity kinematics, and workspace characterization, Siciliano et al. [?] provide detailed algorithms and case studies drawn from industrial manipulators. Their treatment of singularity classification is especially useful for practical applications.

Bullo and Lewis [?] develop geometric control theory on configuration manifolds, extending the ideas of this chapter into the realm of controllability and motion planning on curved spaces. Their framework unifies the holonomic and non-holonomic constraint analysis introduced in Section 3.12 with modern tools from differential geometry.

More broadly, the configuration manifolds studied here are special cases of the smooth manifolds treated in any graduate text on differential geometry. Readers seeking mathematical depth beyond the robotics context will find that the manifold, chart, and atlas concepts introduced in this chapter transfer directly to the general theory.

3.16 Exercises

3.16.1 Foundational Tier

1. A system consisting of a single rigid body in 3D space (not subject to constraints) has how many degrees of freedom? Explain why.

2. For the planar 4-bar linkage, verify Grübler's formula:

   (a) Identify n (movable links), j (joints), and d_i (DOF per joint).

   (b) Compute DOF $= 3n - 3j + \sum d_i$.

   (c) Explain why the result matches your intuition.

3. The circle S^1 is a 1-dimensional manifold. Why can you not cover S^1 with a single chart (U, φ) where $\varphi : U \to \mathbb{R}$? (Hint: think about continuity at the wraparound point.)

4. For a point $q \in S^1$ (the circle), what is the dimension of the tangent space $T_q S^1$? Why?

5. A 2-link planar arm has link lengths $L_1 = 1, L_2 = 1$. At the configuration $q_1 = 0, q_2 = 0$, what is the end-effector position $\mathbf{p}_{end}$? What is the Jacobian at this configuration?

3.16.2 Intermediate Tier

1. Grübler's formula for planar mechanisms: DOF $= 3n - 3j + \sum d_i$.

   (a) A planar slider-crank mechanism has 3 moving links, 4 joints (one revolute, three other). What must the other three joints be to have DOF = 1? Show your calculation.

   (b) Sketch such a mechanism.

2. For the 2-link arm from Example 3.8.1, show that at configuration $q_1 = \pi/2, q_2 = 0$, the Jacobian is singular ($\det J = 0$). What does this singularity represent geometrically? (Hint: what is the end-effector orientation?)

3. Consider the reachable workspace of a 2-link arm with $L_1 = 2, L_2 = 1$.

   (a) What is the inner radius (minimum distance from origin)?

   (b) What is the outer radius (maximum distance)?

   (c) What is the area of the annular workspace?

4. Transition maps on S^1: Using the two charts from Example 3.4.1, show that the transition map τ_{12} is smooth. What would happen if you tried to use a chart (U, φ) with $\varphi(\theta) = \theta$ (angle in radians) covering the entire circle? Why does this fail to be a valid chart?

5. For a unicycle model (Example **??**), write the non-holonomic constraint in Pfaffian form $\mathbf{A}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0}$. How many velocity constraints are there? What is the dimension of the allowed velocity subspace?

3.16.3 Advanced Tier

1. Prove that for a 2-link arm, the workspace is indeed an annulus (not a disk or other shape).

   (a) Show that the reachable set is $\{(x, y) : |L_1 - L_2| \leq r \leq L_1 + L_2\}$ where $r = \sqrt{x^2 + y^2}$.

   (b) Hint: parameterize by $q_1, q_2 \in [0, 2\pi)$ and use the constraint that the sum of two vectors of fixed length L_1, L_2 has magnitude bounded by $L_1 + L_2$ and at least $|L_1 - L_2|$.

2. Inverse Kinematics for a 3-link planar arm: Suppose we augment the 2-link arm with a third link of length $L_3 = 0.5$.

   (a) Write the forward kinematics: x, y, θ in terms of q_1, q_2, q_3.

   (b) If the target is $\mathbf{p}_d = [2.0, 0.5, \pi/4]^T$ (position and orientation), set up the Newton-Raphson iteration $\mathbf{q}_{k+1} = \mathbf{q}_k + J_{kin}^{-1}(\mathbf{q}_k)\mathbf{e}(\mathbf{q}_k)$.

   (c) Numerically solve (using Python or Matlab) starting from an initial guess $\mathbf{q}_0 = [\pi/6, \pi/6, \pi/6]^T$.

3. For a rolling ball on a 2D plane with no-slip condition (Example **??**), write the full state (configuration + velocity) and state the non-holonomic constraint in Pfaffian form. Does this constraint reduce the dimension of the configuration space?

4. Prove the equivalence (Theorem 3.3) by constructing explicit isomorphisms:
   (a) Given a curve $\gamma(t)$ with $\gamma(0) = q$ and local coordinates $\varphi(\gamma(t))$, define a derivation v_γ on smooth functions.
   (b) Conversely, given a derivation v, show you can construct a family of curves whose tangent vectors represent v.
   (c) Verify that these maps are mutual inverses (up to equivalence).
5. Stewart Platform Inverse Kinematics: A Stewart platform has a desired end-effector pose (position $\mathbf{p} \in \mathbb{R}^3$ and orientation $R \in SO(3)$). Given 6 attachment points on the base and 6 on the platform, the inverse kinematics problem is to find the 6 actuator lengths $\ell_1, \ldots, \ell_6$.
   (a) Write the constraint equations (distance constraints).
   (b) Discuss why this nonlinear system is easier to solve than general 6-DOF arm IK (Hint: it is a set of independent distance constraints, not a chain of rotations).
   (c) Sketch an algorithm (e.g., Newton-Raphson or geometric approach) and discuss convergence.